

---

PYTHON · INTERVIEW PREPARATION

# LeetCode Patterns

## The Recitation Method

A pattern book for engineers who have no time to grind. 93 memorisable templates, 1151 high-frequency problems reduced to their smallest kernels.

---

**93** patterns · **1151** interview problems · **Python**

**380** Easy · **641** Medium · **130** Hard

Frequency data drawn from **70** company interview sets

# About This Book

---

LeetCode Patterns: The Recitation Method is organised around *reduction*, not around data structures. Every chapter answers one question: *what is the smallest thing you must remember, such that a whole family of problems becomes a mechanical variation of it?*

The organising principle is deliberately not the textbook taxonomy. A chapter called “BFS” tells you nothing you can memorise. A chapter called “Binary Search — The Boundary Rules” tells you that there are exactly two loop shapes and two midpoint formulas, that `hi = mid` implies the lower midpoint and `lo = mid` implies the upper one, and that violating this pairing is an infinite loop rather than a wrong answer. That is memorisable. That is what this book contains.

## Where the problem set comes from

---

The index is built from 1151 problems that appear in at least one of 70 company interview sets, weighted so that tier-one companies (Google, Amazon, Meta, Microsoft, Apple, Netflix, Bloomberg, ByteDance, Tencent, Alibaba, Baidu) count triple. Difficulty, acceptance rate and topic tags come from LeetCode’s own problem catalogue. The frequency score printed beside each problem is a rough proxy for how often it is asked — not a probability.

## How the numbers were derived

---

Problem identifiers are LeetCode’s *frontend* numbers, the ones that appear in the URL. This distinction matters more than it sounds: LeetCode’s public problem API returns an internal database id in its `question_id` field which diverges from the frontend number (the internal id of problem 704 is 792). Every identifier in this book was resolved through the frontend field and cross-checked against the official title and difficulty, and the build fails loudly if a reference does not resolve.

## A note on premium problems

---

Problems marked with a dagger (†) are LeetCode Premium and are not freely accessible. They are included because they still appear in real interviews and the pattern is worth knowing, but you should not treat them as required practice.

---

This is a sample edition containing the front matter, Chapter 10 and Chapter 5. The full edition contains 93 chapters across 17 parts.

# Contents

---

## **PART II • SEQUENTIAL SCANS**

---

|          |                           |    |
|----------|---------------------------|----|
| <b>5</b> | Two Pointers – Converging | 33 |
|----------|---------------------------|----|

## **PART III • BINARY SEARCH**

---

|           |                                    |   |
|-----------|------------------------------------|---|
| <b>10</b> | Binary Search – The Boundary Rules | 1 |
|-----------|------------------------------------|---|

PART III

# Binary Search

---

Two loop shapes and two midpoint formulas. Everything else is bookkeeping.

# Binary Search - The Boundary Rules

## THE KERNEL

There are only two loop shapes and two midpoint formulas. Pick the shape by asking ‘does the answer live in the closed range [lo, hi] or at the boundary where the range collapses?’

Binary search is the most commonly failed easy problem in technical interviews, and it fails for one reason: candidates memorise *an* implementation rather than the *rule that chooses between* implementations. They write `while lo <= hi`, then pair it with `hi = mid`, and the loop hangs. Or they write `lo = mid` with the lower midpoint and the loop hangs in the other direction. Both bugs come from the same missing piece of knowledge, and both are trivially avoidable once you have it.

This chapter contains no clever ideas. It contains two loop shapes, two midpoint formulas, and one pairing rule that connects them. Everything else in the binary search family — bounds, rotated arrays, peak finding, binary search on the answer — is a substitution into that frame.

Read the boundary rules below, then close the book and write the template from memory. If you cannot, you have not yet learned this chapter, regardless of how many problems you have solved.

**55**PROBLEMS  
MAPPED**21**

EASY

**31**

MEDIUM

**3**

HARD

## The Boundary Rules

---

These ten rules are the whole chapter. Rules 1 through 5 are the framework; rules 6 through 10 are the traps.

- 1 **Shape A — closed range.** `while lo <= hi:` with `hi = mid - 1` and `lo = mid + 1`. Exits with `lo == hi + 1`. Use when a target may be absent and you must report 'not found'.
- 2 **Shape B — half-open, answer is the collapse point.** `while lo < hi:` with `hi = mid` and `lo = mid + 1`. Exits with `lo == hi == answer`. Use for lower bound, upper bound, 'smallest feasible', and binary search on the answer.
- 3 **Midpoint rule.** `mid = lo + (hi - lo) // 2` is the lower mid.  
`mid = lo + (hi - lo + 1) // 2` is the upper mid.
- 4 **The pairing rule — memorise this one line.** If the branch that moves `lo` can leave `lo == mid` (i.e. `lo = mid`), you MUST use the upper mid. If the branch that moves `hi` can leave `hi == mid` (i.e. `hi = mid`), you MUST use the lower mid. Violating this is an infinite loop, not a wrong answer — which is why it is easy to spot under test.
- 5 Equivalently: `hi = mid` implies lower mid; `lo = mid` implies upper mid.
- 6 **Return rule.** After Shape A the candidate is `lo` and it is valid only if `lo < n` and `a[lo] == target`. After Shape B `lo` is the answer unconditionally — Shape B never returns 'not found', it returns the insertion point, which is why it is the shape you want for bounds.
- 7 **Overflow.** `lo + (hi - lo) // 2` never overflows; `(lo + hi) // 2` can in a fixed-width language. Python has arbitrary precision, but write the safe form anyway — interviewers read it as a signal.
- 8 **Never `while lo <= hi` with `hi = mid`.** The range does not shrink when `hi == mid == lo + 1`, and you loop forever.
- 9 **Never `lo = mid` without `+1`.** Same failure, other side.
- 10 Searching a 2D matrix that is sorted row-wise and column-wise ( 240 ) is NOT a binary search over the whole matrix — start at the top-right corner and walk, because that corner is the only cell that is a minimum in one direction and a maximum in the other.

## The Template

There are exactly three code shapes. Shape A is for ‘find this exact value and tell me if it is missing’. Shape B is for ‘find the boundary where a predicate flips’. Shape C is Shape B’s mirror, and it exists only because of the midpoint rule: when the branch that moves `lo` can leave `lo == mid`, the midpoint must round up.

Memorise Shape B and Shape C as a pair. If you can write both from memory and state why Shape C needs the `+ 1`, you can solve every problem in this chapter and the next four.

```
# ----- Shape A
# Closed range [lo, hi]. Exits with lo == hi + 1.
# Use when the target may be ABSENT and you must report that.

def search_closed(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:                # <= : the range is closed
        mid = lo + (hi - lo) // 2  # lower mid
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1            # +1 : mid is excluded
        else:
            hi = mid - 1           # -1 : mid is excluded
    return -1                      # absent; lo is the insert point

# ----- Shape B
# Half-open [lo, hi). The answer is where the range collapses.
# "Smallest i such that predicate(i) is True."

def lower_bound(a, x):
    lo, hi = 0, len(a)            # hi is one PAST the end
    while lo < hi:                # < : the range is half-open
        mid = lo + (hi - lo) // 2  # lower mid
        if a[mid] >= x:
            hi = mid               # mid may be the answer: keep it
        else:
            lo = mid + 1           # mid is not: exclude it
    return lo                     # lo == hi == the answer

# ----- Shape C
```

```

# Shape B's mirror. "Largest x with feasible(x) True."
# The midpoint MUST round up: lo = mid has to make progress.

def largest_feasible(lo, hi, feasible):
    while lo < hi:
        mid = lo + (hi - lo + 1) // 2  # UPPER mid (lo = mid)
        if feasible(mid):
            lo = mid  # mid may be the answer: keep it
        else:
            hi = mid - 1  # mid is not: exclude it
    return lo  # the largest feasible value

# ----- the one-line form
import bisect

def count_equal(a, x):
    """How many elements equal x? Two Shape-B calls."""
    return bisect.bisect_right(a, x) - bisect.bisect_left(a, x)

```

## Why Shape B cannot return “not found”

This surprises people, so it is worth being explicit. Shape B narrows a half-open range until `lo == hi`, and returns `lo`. That value always exists — it is an index in `[0, n]`. When the target is absent, Shape B returns the *insertion point*, which is the index where the target would go to keep the array sorted. That is not a failure; it is extra information, and it is exactly what `Search Insert Position` (35) asks for.

So Shape B is strictly more informative than Shape A, and you should prefer it whenever the problem asks for a position rather than a yes/no. When the problem genuinely needs ‘not found’, compute the lower bound and then verify: `i = bisect_left(a, x)` and check `i < len(a)` and `a[i] == x`.

### THE REFRAMING

Shape B never fails. It returns the insertion point, which is the answer to a different question than the one you thought you were asking.

## The pairing rule, stated three ways

---

The infinite-loop bug always has the same cause: a branch that assigns `mid` to a bound without moving past it, combined with a midpoint that rounds the wrong way. Three equivalent statements of the fix:

- **By the branch:** if a branch assigns `lo = mid`, use the upper mid. If a branch assigns `hi = mid`, use the lower mid.
- **By the loop:** `while lo < hi` with `hi = mid` is the lower-mid shape. `while lo < hi` with `lo = mid` is the upper-mid shape. Never mix them.
- **By the symptom:** if your search hangs, your midpoint rounds down and your `lo` branch does not advance. Add `+ 1` to the midpoint — do not add `+ 1` to `lo = mid`.

## Choosing between the shapes in an interview

---

Say the following sentence out loud before you write code: *the answer is either a value I am looking for, in which case I use the closed shape and check for absence; or it is the boundary where a monotone predicate flips, in which case I use the half-open shape and return the collapse point.*

That sentence takes four seconds and it converts the most error-prone part of the interview into a decision you have already made.

## Worked Solutions

---

Ten problems, in increasing order of how much the boundary rules matter. Each one is annotated with the line of the template that changes.

**704 Binary Search**

LC 704 · Easy · asked by 9 companies · AC 61.4% · Array, Binary Search

**THE PROBLEM**

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with  $O(\log n)$  runtime complexity.

**Example 1:**

**Input:** `nums = [-1,0,3,5,9,12]`, `target = 9`  
**Output:** `4`

**Explanation** 9 exists in `nums` and its index is 4

**Example 2:**

**Input:** `nums = [-1,0,3,5,9,12]`, `target = 2`  
**Output:** `-1`

**Explanation** 2 does not exist in `nums` so return -1

**Constraints:**

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 < \text{nums}[i], \text{target} < 10^4$
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

**SIGNATURE**

```
def search(nums, target): # -> integer
```

**IN ONE LINE** Return the index of `target` in a sorted array, or `-1`.

**THE INSIGHT** This is the pure Shape A. The only decision is that the range is closed, so the loop uses `<=` and both branches move past `mid`.

The closed range `[lo, hi]` contains the candidates. Each iteration examines `mid` and discards it by moving a bound past it. When `lo > hi` the range is empty and the target is absent.

The invariant to state: **every index outside `[lo, hi]` has been ruled out**. That is what justifies returning `-1` at the end.

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

**Complexity.**  $O(\log n)$  time,  $O(1)$  space. The loop runs at most `ceil(log2(n + 1))` times.

#### PITFALL

Writing `hi = mid` instead of `hi = mid - 1`. With the lower midpoint this hangs on two-element ranges: `lo = 0, hi = 1` gives `mid = 0`, and `hi = 0` repeats forever.

#### PITFALL

Writing `while lo < hi`. That drops the last candidate and silently returns `-1` for a single-element array.

**Follow-up.** "What if the array has duplicates and you want the first occurrence?" That is chapter 11 – `bisect_left`, which is Shape B with the predicate `a[i] >= x`.

## 35 Search Insert Position

LC 35 · Easy · asked by 9 companies · AC 52.3% · Array, Binary Search

### THE PROBLEM

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with  $O(\log n)$  runtime complexity.

**Example 1:**

**Input:** nums = [1,3,5,6], target = 5  
**Output:** 2

**Example 2:**

**Input:** nums = [1,3,5,6], target = 2  
**Output:** 1

**Example 3:**

**Input:** nums = [1,3,5,6], target = 7  
**Output:** 4

**Constraints:**

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` contains **distinct** values sorted in **ascending** order.
- $-10^4 \leq \text{target} \leq 10^4$

**SIGNATURE**

```
def searchInsert(nums, target): # -> integer
```

**IN ONE LINE** Return the index where `target` should be inserted to keep the array sorted.

**THE INSIGHT** This is Shape B with no modification at all. The insertion point *is* the collapse point of the half-open range.

Compare with 704: there, an absent target was a failure. Here it is the answer. The same search returns both.

`bisect.bisect_left(nums, target)` is the standard library version. Write the loop by hand in an interview, then mention that `bisect_left` does exactly this.

```
def search_insert(nums, target):
    lo, hi = 0, len(nums)          # half-open: hi may be len(nums)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] >= target:    # predicate: a[mid] >= target
            hi = mid               # mid may be the answer
        else:
            lo = mid + 1           # mid is not
    return lo                      # == bisect_left(nums, target)
```

**Complexity.**  $O(\log n)$  time,  $O(1)$  space. Note `hi` starts at `len(nums)`, which is why the range is half-open.

#### PITFALL

Starting `hi = len(nums) - 1`. Then a target larger than every element returns `len(nums) - 1` instead of `len(nums)`.

#### PITFALL

Using `nums[mid] > target` as the predicate. That computes `bisect_right`, which is off by one when duplicates exist.

**Follow-up.** “And if you want the position *after* all equal elements?” Change `>=` to `>`. The two functions differ by exactly one character – which is why `count_equal` in the template is a subtraction of two `bisect` calls rather than a custom loop.

## 34 Find First and Last Position of Element in Sorted Array

LC 34 · **Medium** · asked by 0 companies · AC 49.7% · Array, Binary Search

### THE PROBLEM

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with  $O(\log n)$  runtime complexity.

**Example 1:**

**Input:** nums = [5,7,7,8,8,10], target = 8  
**Output:** [3,4]

**Example 2:**

**Input:** nums = [5,7,7,8,8,10], target = 6  
**Output:** [-1,-1]

**Example 3:**

**Input:** nums = [], target = 0  
**Output:** [-1,-1]

**Constraints:**

- `0 <= nums.length <= 105`
- `-109 <= nums[i] <= 109`
- `nums` is a non-decreasing array.
- `-109 <= target <= 109`

**SIGNATURE**

```
def searchRange(nums, target): # -> integer[]
```

**IN ONE LINE** Return `[first, last]` index of `target`, or `[-1, -1]`.

**THE INSIGHT** Two `bisect` calls with predicates `>=` and `>`. No custom binary search is needed, and no post-processing beyond one validity check.

`first = bisect_left(a, x)` is the smallest index with `a[i] >= x`. `last = bisect_right(a, x) - 1` is the largest index with `a[i] <= x`.

The existence check is `first < len(a)` and `a[first] == x`. Without it, an absent target returns `[n, n-1]`, which is nonsense that some test cases will catch.

```
from bisect import bisect_left, bisect_right

def search_range(nums, target):
    first = bisect_left(nums, target) # smallest index, a[i] >= x
    if first == len(nums) or nums[first] != target:
        return [-1, -1] # absent; Shape B cannot tell
    last = bisect_right(nums, target) - 1 # largest index, a[i] <= x
```

```

    return [first, last]

# By hand, to show it is the same shape twice:
def search_range_manual(nums, target):
    def bound(x, strict):
        lo, hi = 0, len(nums)
        while lo < hi:
            mid = lo + (hi - lo) // 2
            if nums[mid] > x or (not strict and nums[mid] == x):
                hi = mid
            else:
                lo = mid + 1
        return lo
    first = bound(target, False)
    if first == len(nums) or nums[first] != target:
        return [-1, -1]
    return [first, bound(target, True) - 1]

```

**Complexity.**  $O(\log n)$  time,  $O(1)$  space – two searches, so about twice the constant of 704.

#### PITFALL

Returning `[first, bisect_right(nums, target)]`. The right bound is *exclusive*, so it needs the `- 1`.

#### PITFALL

Skipping the validity check. `[n, n-1]` is not a valid answer and will fail the empty-result test.

**Follow-up.** “What if you must not use `bisect`?” Write `bound` once with a `strict` flag, as above. The flag is the only difference between the two calls.

## 278 First Bad Version

LC 278 · Easy · asked by 7 companies · AC 47.7% · Binary Search, Interactive

### THE PROBLEM

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each

version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have  $n$  versions  $[1, 2, \dots, n]$  and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether `version` is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

#### Example 1:

**Input:**  $n = 5$ ,  $\text{bad} = 4$   
**Output:** 4

#### Explanation

```
call isBadVersion(3) -> false
call isBadVersion(5) -> true
call isBadVersion(4) -> true
Then 4 is the first bad version.
```

#### Example 2:

**Input:**  $n = 1$ ,  $\text{bad} = 1$   
**Output:** 1

#### Constraints:

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$

#### SIGNATURE

```
def firstBadVersion(n, bad): # -> integer
```

**IN ONE LINE** Given `isBadVersion(v)`, find the first bad version.

**THE INSIGHT** The predicate `isBadVersion` is monotone by construction (false, false, ..., true, true), so this is Shape B with no array at all.

This is the cleanest possible demonstration of the pattern: there is no data structure, only a monotone predicate. The search space is the version numbers, not an index into anything.

Set `lo = 1`, `hi = n` and let `hi` be *inclusive* of the answer candidate. `while lo < hi` with `hi = mid` and `lo = mid + 1` converges on the first `true`.

```
def first_bad_version(n):
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if isBadVersion(mid):      # monotone predicate
            hi = mid                # mid may be the first bad one
        else:
            lo = mid + 1            # mid is definitely good
    return lo
```

**Complexity.**  $O(\log n)$  calls to `isBadVersion`,  $O(1)$  space. The number of calls is the cost model here – say so, because the problem statement asks for it.

### PITFALL

Using `Shape A` and a `last_bad` variable. That is  $O(n)$  in the worst case and misses the point of the exercise.

### PITFALL

Writing `hi = mid - 1`. With `hi` inclusive of a candidate, subtracting 1 discards the answer.

**Follow-up.** “How many calls does your solution make?” `ceil(log2 n)`. For `n = 231 - 1` that is 31 calls. Being able to answer this precisely is worth as much as the code.

## 69 Sqrt(x)

LC 69 · **Easy** · asked by 7 companies · AC 42.3% · Math, Binary Search, Newton's Method

### THE PROBLEM

Given a non-negative integer `x`, return the square root of `x` rounded down to the nearest integer. The returned integer should be **non-negative** as well.

You **must not use** any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

### Example 1:

**Input:** `x = 4`  
**Output:** `2`

**Explanation** The square root of 4 is 2, so we return 2.

### Example 2:

**Input:** `x = 8`

**Output:** `2`

**Explanation** The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is returned.

### Constraints:

- `0 <= x <= 231 - 1`

### SIGNATURE

```
def mySqrt(x):    # -> integer
```

**IN ONE LINE** Integer square root of `x`, rounded down.

**THE INSIGHT** This is Shape C — the *largest feasible* form — and it is the problem that exposes the midpoint rule.

The predicate is `mid * mid <= x`. It is monotone: if `mid` works, every smaller value works. So the answer is the largest feasible `mid`, which is Shape C.

Because the feasible branch assigns `lo = mid`, the midpoint must round **up**. If you use the lower midpoint here, `lo = mid` makes no progress on a two-element range and the loop hangs. This is the bug the whole chapter is about, in its purest form.

Note `hi = x // 2 + 1` rather than `x`. For `x >= 4` the answer is at most `x // 2`, so this halves the search range; the `+ 1` keeps `x = 1` working.

```
def my_sqrt(x):
    if x < 2:
        return x
    lo, hi = 1, x // 2 + 1          # answer <= x // 2 for x >= 4
    while lo < hi:
        mid = lo + (hi - lo + 1) // 2  # UPPER mid: required by lo = mid
        if mid * mid <= x:             # feasible
            lo = mid                  # keep it; mid may be the answer
        else:
            hi = mid - 1              # discard it
```

```

    return lo

# Same thing without the midpoint trap; Shape B on first failure:
def my_sqrt_shapeb(x):
    lo, hi = 1, x // 2 + 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if mid * mid > x:                # "too big" is the predicate
            hi = mid
        else:
            lo = mid + 1
    return lo - 1                        # one before first too-big value

```

**Complexity.**  $O(\log x)$  time,  $O(1)$  space. Newton's method is faster in practice but is not the expected answer here.

#### PITFALL

Using the lower midpoint with `lo = mid`. **Infinite loop.** If you remember only one thing from this chapter, remember this line.

#### PITFALL

Forgetting the `x < 2` guard. `x = 0` gives `hi = 1` and `x = 1` gives `hi = 1`, both of which happen to work, but the guard makes it obvious rather than accidental.

**Follow-up.** "Can you avoid the midpoint rule entirely?" Yes – search for the first `mid` with `mid * mid > x` using Shape B and return `lo - 1`, as in `my_sqrt_shapeb`. **Choosing the predicate so that Shape B applies is a legitimate way to dodge the upper midpoint**, and offering it shows you understand the trade.

## 162 Find Peak Element

LC 162 · Medium · asked by 8 companies · AC 47.2% · Array, Binary Search

### THE PROBLEM

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in  $O(\log n)$  time.

#### Example 1:

**Input:** `nums = [1,2,3,1]`

**Output:** `2`

**Explanation** 3 is a peak element and your function should return the index number 2.

#### Example 2:

**Input:** `nums = [1,2,1,3,5,6,4]`

**Output:** `5`

**Explanation** Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

#### Constraints:

- `1 <= nums.length <= 1000`
- `-231 <= nums[i] <= 231 - 1`
- `nums[i] != nums[i + 1]` for all valid `i`.

#### SIGNATURE

```
def findPeakElement(nums): # -> integer
```

**IN ONE LINE** Find any peak element in an array where `a[i] != a[i+1]`.

**THE INSIGHT** The array is not sorted, but the *comparison with the right neighbour* is a monotone predicate. Where the slope turns from up to down, there is a peak.

Define `predicate(mid) = a[mid] > a[mid + 1]`. If `mid` is on a downward slope, a peak exists at or to the left. If `mid` is on an upward slope, a peak exists at or to the right.

That makes the predicate monotone in the search sense — false, false, ..., true, true — so Shape B applies directly. `hi` starts at `len(a) - 1` because `mid + 1` must be in range.

This is the template for a whole family: **binary search works whenever you can decide which half contains the answer, even if the array is not sorted.**

```
def find_peak_element(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] > nums[mid + 1]: # descending -> peak at or left
            hi = mid
        else: # ascending -> peak at or right
            lo = mid + 1
    return lo # a peak index
```

**Complexity.**  $O(\log n)$  time,  $O(1)$  space. The problem asks for  $O(\log n)$ , which is the hint that this is a binary search despite the unsorted input.

#### PITFALL

Comparing with `a[mid - 1]` instead. Both work, but mixing the two comparisons across the two branches is how off-by-one bugs get in. Pick one reference and stay consistent.

#### PITFALL

Reading `nums[mid + 1]` when `mid` could be `len(nums) - 1`. With `hi = len(nums) - 1` and the lower midpoint, `mid < hi` always holds, so `mid + 1` is safe — but you must be able to say why.

**Follow-up.** “What if there are duplicates?” [Find Peak Element II](#) (1901) does it in 2-D. With duplicates in 1-D the predicate stops being monotone and you need [Find Minimum in Rotated Sorted Array II](#) (154)'s trick of shrinking both ends.

## 33 Search in Rotated Sorted Array

LC 33 · **Medium** · asked by 21 companies · AC 45.6% · Array, Binary Search

### THE PROBLEM

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` ( $1 \leq k < \text{nums.length}$ ) such that the resulting array is

`[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with  $O(\log n)$  runtime complexity.

#### Example 1:

**Input:** `nums = [4,5,6,7,0,1,2]`, `target = 0`  
**Output:** `4`

#### Example 2:

**Input:** `nums = [4,5,6,7,0,1,2]`, `target = 3`  
**Output:** `-1`

#### Example 3:

**Input:** `nums = [1]`, `target = 0`  
**Output:** `-1`

#### Constraints:

- `1 <= nums.length <= 5000`
- `-104 <= nums[i] <= 104`
- All values of `nums` are **unique**.
- `nums` is an ascending array that is possibly rotated.
- `-104 <= target <= 104`

#### SIGNATURE

```
def search(nums, target): # -> integer
```

**IN ONE LINE** Search for `target` in a rotated sorted array with distinct values.

**THE INSIGHT** Exactly one of the two halves `[lo, mid]` and `[mid, hi]` is sorted. Find which, then ask whether the target lies inside that sorted half.

The array is not sorted, but it is *piecewise* monotone, and one piece always contains `mid`. The test `a[lo] <= a[mid]` tells you whether the left half is the sorted one.

If the left half is sorted, the target is in it exactly when `a[lo] <= target < a[mid]`; otherwise it must be in the right half. The two branches are mirror images.

This is Shape A with a two-step decision replacing the three-way comparison. The bounds still move past `mid`.

```
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:           # left half [lo, mid] sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1               # target lies inside it
            else:
                lo = mid + 1               # so it is in the right half
        else:                             # right half [mid, hi] sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1               # target lies inside it
            else:
                hi = mid - 1               # so it is in the left half
    return -1
```

**Complexity.**  $O(\log n)$  time,  $O(1)$  space. The rotation costs nothing because the sorted-half test is  $O(1)$ .

#### PITFALL

Using `nums[lo] < nums[mid]` instead of `<=`. With two elements `lo == mid`, and the strict comparison misclassifies the half.

#### PITFALL

Testing `nums[lo] <= target <= nums[mid]` with `<=` on the right end. `nums[mid] == target` was already handled, and including it again makes the `else` branch wrong.

#### PITFALL

Deciding which half is sorted by comparing with `a[hi]` in one branch and `a[lo]` in the other. Both are valid references, but you must be consistent — and the `a[hi]` reference is the one you need for problem 153, so pick that and reuse it.

**Follow-up.** "What if values can repeat?" Problem 81. When `a[lo] == a[mid] == a[hi]` you cannot tell which half is sorted, so you must shrink by one from each end and accept  $O(n)$  worst-case time. Say that the guarantee is lost – that is the expected answer.

## 153 Find Minimum in Rotated Sorted Array

LC 153 · Medium · asked by 7 companies · AC 55.3% · Array, Binary Search

### THE PROBLEM

Suppose an array of length `n` sorted in ascending order is **rotated** between `1` and `n` times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated `4` times.
- `[0,1,2,4,5,6,7]` if it was rotated `7` times.

Notice that **rotating** an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of **unique** elements, return *the minimum element of this array*.

You must write an algorithm that runs in  $O(\log n)$  time.

#### Example 1:

**Input:** `nums = [3,4,5,1,2]`  
**Output:** `1`

**Explanation** The original array was `[1,2,3,4,5]` rotated 3 times.

#### Example 2:

**Input:** `nums = [4,5,6,7,0,1,2]`  
**Output:** `0`

**Explanation** The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

#### Example 3:

**Input:** `nums = [11,13,15,17]`  
**Output:** `11`

**Explanation** The original array was `[11,13,15,17]` and it was rotated 4 times.

#### Constraints:

- `n == nums.length`

- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- All the integers of `nums` are **unique**.
- `nums` is sorted and rotated between `1` and `n` times.

**SIGNATURE**

```
def findMin(nums):    # -> integer
```

**IN ONE LINE** Find the minimum in a rotated sorted array with distinct values.

**THE INSIGHT** Compare `a[mid]` with `a[hi]`. If `a[mid] < a[hi]` the right half is sorted, so the minimum is at or left of `mid`. Otherwise it is strictly to the right.

This is the cleanest rotated-array search because it needs no nested decision: the single comparison with `a[hi]` partitions the array correctly every time.

Note that `hi = mid` (not `mid - 1`) — `mid` may be the minimum, so it must stay in the range. That makes this Shape B.

The comparison reference matters here. Comparing with `a[lo]` instead does not work for the minimum, because when `a[lo] < a[mid]` you still cannot tell whether the rotation point is before or after `mid`. **`a[hi]` is the right reference for minimum-finding; remember that as a separate fact from problem 33.**

```
def find_min(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] < nums[hi]:    # right half is sorted
            hi = mid                # minimum is at or left of mid
        else:                      # minimum is strictly right of mid
            lo = mid + 1
    return nums[lo]
```

**Complexity.**  $O(\log n)$  time,  $O(1)$  space.

**PITFALL**

Writing `hi = mid - 1` in the first branch. If `mid` is the minimum you have just discarded it.

**PITFALL**

Returning `nums[hi]` at the end. Both work since `lo == hi`, but pick one and be consistent with problem 154.

**Follow-up.** “With duplicates?” Problem 154 adds `elif nums[mid] == nums[hi]: hi -= 1`. The `elif` must come *after* the `<` test, and the `-= 1` is what degrades the worst case to  $O(n)$  when the array is all equal values.

**540 Single Element in a Sorted Array**

LC 540 · **Medium** · asked by 6 companies · AC 59.4% · Array, Binary Search

**THE PROBLEM**

You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once.

Return *the single element that appears only once*.

Your solution must run in  $O(\log n)$  time and  $O(1)$  space.

**Example 1:**

**Input:** `nums = [1,1,2,3,3,4,4,8,8]`  
**Output:** `2`

**Example 2:**

**Input:** `nums = [3,3,7,7,10,11,11]`  
**Output:** `10`

**Constraints:**

- `1 <= nums.length <= 105`
- `0 <= nums[i] <= 105`

**SIGNATURE**

```
def singleNonDuplicate(nums): # -> integer
```

**IN ONE LINE** In a sorted array where every element appears twice except one, find the single element.

**THE INSIGHT** Binary search on the *parity of the index*. Before the single element, pairs start at even indices; after it, at odd indices. The index XOR trick makes the pair test one expression.

For any index `i`, its pair partner is `i ^ 1`. Before the single element, `a[i] == a[i ^ 1]` holds for every `i`; after it, the parity is flipped and the equality fails.

So the predicate `a[mid] == a[mid ^ 1]` is monotone: true, true, ..., false, false. Shape B finds the first `false`.

`mid` must be even for the test to mean what you think. `mid -= mid % 2` forces that, and it is the line that makes the solution robust rather than lucky.

```
def single_non_duplicate(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        mid -= mid % 2           # force mid to be even
        if nums[mid] == nums[mid + 1]: # pair intact -> single right
            lo = mid + 2
        else:                     # broken pair -> single at or left
            hi = mid
    return nums[lo]
```

**Complexity.**  $O(\log n)$  time,  $O(1)$  space. The XOR form `mid ^ 1` is equivalent and slightly more elegant; both are accepted.

#### PITFALL

Advancing `lo = mid + 1`. Because pairs move in steps of two, the correct advance is `mid + 2` — a `+ 1` leaves `lo` pointing at the second half of a pair.

#### PITFALL

Forgetting the `mid -= mid % 2` normalisation. On odd `mid` the comparison tests the wrong pair and the search goes to the wrong half.

**Follow-up.** “Where else does index-parity binary search appear?” Any problem where a *structural property* flips at a point — `Find the Duplicate Number` (287) has a similar flavour, though its expected answer is Floyd’s cycle detection from chapter 35.

**74 Search a 2D Matrix**LC 74 · **Medium** · asked by 10 companies · AC 54.3% · Array, Binary Search, Matrix**THE PROBLEM**

You are given an  $m \times n$  integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` if `target` is in `matrix` or `false` otherwise.

You must write a solution in  $O(\log(m * n))$  time complexity.

**Example 1:**

[diagram omitted — see [leetcode.com/problems/search-a-2d-matrix/](https://leetcode.com/problems/search-a-2d-matrix/)]

**Input:** `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3`  
**Output:** `true`

**Example 2:**

[diagram omitted — see [leetcode.com/problems/search-a-2d-matrix/](https://leetcode.com/problems/search-a-2d-matrix/)]

**Input:** `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 13`  
**Output:** `false`

**Constraints:**

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 100`
- `-104 <= matrix[i][j], target <= 104`

**SIGNATURE**

```
def searchMatrix(matrix, target): # -> boolean
```

**IN ONE LINE** Search a matrix whose rows are sorted and whose first element of each row is greater than the last element of the previous row.

**THE INSIGHT** The matrix is a sorted array that has been wrapped. Treat it as a flat array of length `m * n` and convert the midpoint with `divmod`.

The problem statement guarantees a *total* order, which is stronger than the sorted-matrix condition. That total order is exactly what lets you index the matrix as a flat array.

`row, col = divmod(mid, n)` converts a flat index to a cell. Everything else is problem 704 verbatim.

**Contrast with problem 240**, whose matrix is sorted only row-wise and column-wise. There the flat trick fails and you must walk from the top-right corner instead. Knowing which of the two matrices you have is the entire decision.

```
def search_matrix(matrix, target):
    m, n = len(matrix), len(matrix[0])
    lo, hi = 0, m * n - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        row, col = divmod(mid, n)    # flat index -> cell
        if matrix[row][col] == target:
            return True
        if matrix[row][col] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return False
```

**Complexity.**  $O(\log(mn))$  time,  $O(1)$  space.

#### PITFALL

Writing `matrix[mid // n][mid % n]` inline four times. `divmod` once per iteration is clearer and no slower.

#### PITFALL

Using this solution on problem 240's matrix. It produces wrong answers, not an error — which is why the distinction is worth stating explicitly in an interview.

**Follow-up.** “And if the matrix is only row-wise and column-wise sorted?” Start at the top-right corner. If the value is larger than the target, move left; if smaller, move down.  $O(m + n)$ , and it is the only cell from which both moves are informative.

## The Index – What Changes in the Template

Every remaining problem in this chapter, sorted by how often it is asked. The final column names the single thing that changes in the template — read that column and nothing else on a first pass.

| #    | PROBLEM                                         | LEVEL  | ASKED | WHAT CHANGES                                                                                                     |
|------|-------------------------------------------------|--------|-------|------------------------------------------------------------------------------------------------------------------|
| 33   | Search in Rotated Sorted Array                  | Medium | 21    |                                                                                                                  |
| 4    | Median of Two Sorted Arrays                     | Hard   | 13    | Shape B on the partition of the <b>shorter</b> array; <code>±inf</code> sentinels remove every edge case (ch 14) |
| 74   | Search a 2D Matrix                              | Medium | 10    | Flatten: <code>row, col = divmod(mid, n)</code> – total order makes it 1-D                                       |
| 1146 | Snapshot Array                                  | Medium | 2     | Per-index <code>((snap_id, value))</code> history + <code>bisect_right</code> (ch 24)                            |
| 1539 | Kth Missing Positive Number                     | Easy   | 5     | Shape B: missing count at <code>mid</code> is <code>a[mid] - mid - 1 &gt;= k</code>                              |
| 153  | Find Minimum in Rotated Sorted Array            | Medium | 7     |                                                                                                                  |
| 240  | Search a 2D Matrix II                           | Medium | 6     | Not binary search – walk from the top-right corner, $O(m+n)$                                                     |
| 1428 | Leftmost Column with at Least a One †           | Medium | 3     | 240's walk, but only left/down moves – never right                                                               |
| 81   | Search in Rotated Sorted Array II               | Medium | 4     | Shape A, but <code>a[lo] == a[mid] == a[hi]</code> means shrink both ends; $O(n)$ worst case                     |
| 1287 | Element Appearing More Than 25% In Sorted Array | Easy   | 2     | Binary search the boundary of the majority run                                                                   |
| 744  | Find Smallest Letter Greater Than Target        | Easy   | 3     | <code>bisect_right</code> ; wrap to <code>a[0]</code> when the result is past the end                            |
| 2089 | Find Target Indices After Sorting Array         | Easy   | 2     | Sort, then <code>bisect_left / bisect_right</code> to get the range                                              |
| 888  | Fair Candy Swap                                 | Easy   | 2     | Set difference on the two sums – no search needed                                                                |
| 154  | Find Minimum in Rotated Sorted Array II         | Hard   | 1     | Shape B vs <code>a[hi]</code> ; add <code>elif a[mid] == a[hi]: hi -= 1</code>                                   |

| #    | PROBLEM                                                                  | LEVEL  | ASKED | WHAT CHANGES                                                                                   |
|------|--------------------------------------------------------------------------|--------|-------|------------------------------------------------------------------------------------------------|
| 1237 | Find Positive Integer Solution for a Given Equation                      | Medium | 1     | Monotone 2-D walk, same as 240                                                                 |
| 1346 | Check If N and Its Double Exist                                          | Easy   | 1     | Hash set; or sort and search for <code>2x</code>                                               |
| 1351 | Count Negative Numbers in a Sorted Matrix                                | Easy   | 1     | <code>bisect_left</code> per row, or the 240 staircase walk                                    |
| 275  | H-Index II                                                               | Medium | 1     | Shape B: first <code>i</code> with <code>citations[i] &gt;= n - i</code>                       |
| 1385 | Find the Distance Value Between Two Arrays                               | Easy   | 1     | Sort <code>b</code> , <code>bisect_left</code> , then check both neighbours                    |
| 1337 | The K Weakest Rows in a Matrix                                           | Easy   | 1     | <code>bisect_left</code> per row for the soldier count, then sort by <code>(count, row)</code> |
| 1608 | Special Array With X Elements Greater Than or Equal X                    | Easy   | 1     | Sort, then Shape B over <code>x</code> , or a bucket count                                     |
| 2080 | Range Frequency Queries                                                  | Medium | 1     | <code>bisect_left / bisect_right</code> over per-value index lists                             |
| 441  | Arranging Coins                                                          | Easy   | 0     | Shape B on <code>k(k+1)/2 &lt;= n</code> , or close the quadratic form                         |
| 702  | Search in a Sorted Array of Unknown Size †                               | Medium | 0     | Shape A with <code>hi</code> doubling until it exceeds the target, then search                 |
| 1064 | Fixed Point †                                                            | Easy   | 0     | Shape B: first <code>i</code> with <code>a[i] &gt;= i</code>                                   |
| 1292 | Maximum Side Length of a Square with Sum Less than or Equal to Threshold | Medium | 0     | 2-D prefix sums + binary search the side length (ch 13, 16)                                    |
| 1631 | Path With Minimum Effort                                                 | Medium | 0     | Binary search the effort + BFS feasibility (ch 13)                                             |
| 1855 | Maximum Distance Between a Pair of Values                                | Medium | 0     | Two pointers, not binary search – <code>i</code> and <code>j</code> both advance               |
| 1885 | Count Pairs in Two Arrays †                                              | Medium | 0     | Sort + <code>bisect</code> per element                                                         |
| 1898 | Maximum Number of Removable Characters                                   | Medium | 0     | Binary search the number removed + a subsequence check (ch 13)                                 |
| 1979 | Find Greatest Common Divisor of Array                                    | Easy   | 0     | <code>math.gcd(min(a), max(a))</code>                                                          |

| #    | PROBLEM                                                   | LEVEL  | ASKED | WHAT CHANGES                                                     |
|------|-----------------------------------------------------------|--------|-------|------------------------------------------------------------------|
| 2048 | Next Greater Numerically Balanced Number                  | Medium | 0     | Enumerate digit patterns – no search involved                    |
| 2141 | Maximum Running Time of N Computers                       | Hard   | 0     | Binary search the runtime + a feasibility sum (ch 13)            |
| 2226 | Maximum Candies Allocated to K Children                   | Medium | 0     | Binary search the pile size + greedy count (ch 13)               |
| 2300 | Successful Pairs of Spells and Potions                    | Medium | 0     | Sort potions, <code>bisect_left</code> per spell                 |
| 2387 | Median of a Row Wise Sorted Matrix †                      | Medium | 0     | Binary search the value + per-row count (ch 14)                  |
| 2476 | Closest Nodes Queries in a Binary Search Tree             | Medium | 0     | Inorder to a sorted array, then <code>bisect</code>              |
| 2529 | Maximum Count of Positive Integer and Negative Integer    | Easy   | 0     | <code>bisect_left(0)</code> and <code>n - bisect_right(0)</code> |
| 2540 | Minimum Common Value                                      | Easy   | 0     | Two pointers over two sorted arrays                              |
| 2563 | Count the Number of Fair Pairs                            | Medium | 0     | Sort + <code>bisect</code> per element                           |
| 2592 | Maximize Greatness of an Array                            | Medium | 0     | Sort + two pointers                                              |
| 2772 | Apply Operations to Make All Array Elements Equal to Zero | Medium | 0     | Difference array – no binary search (ch 17)                      |
| 2845 | Count of Interesting Subarrays                            | Medium | 0     | Prefix sums + hash (ch 15)                                       |
| 2981 | Find Longest Special Substring That Occurs Thrice I       | Medium | 0     | Enumerate candidate lengths and count                            |
| 3020 | Find the Maximum Number of Elements in Subset             | Medium | 0     | Enumerate powers of two and count                                |

## Pitfalls

- 1 **The infinite loop.** `lo = mid` with the lower midpoint, or `hi = mid` with the upper midpoint. Both hang. The fix is always to change the midpoint, never to change the assignment.
- 2 **while `lo <= hi` with `hi = mid`.** The range does not shrink when `hi == mid == lo + 1`, so this hangs even though both individual lines look correct. The loop condition and the bound assignments are a matched pair; change them together.
- 3 **Forgetting that `bisect_right` is exclusive.** `bisect_right(a, x) - 1` is the last index of `x`, not `bisect_right(a, x)`.
- 4 **Assuming `bisect` means 'found'.** Both `bisect_left` and `bisect_right` return a valid index even when the value is absent. The existence check `i < n` and `a[i] == x` is always your responsibility.
- 5 **Comparing with the wrong reference in a rotated array.** `a[lo]` is the right reference for *searching* (problem 33); `a[hi]` is the right reference for *finding the minimum* (problem 153). Mixing them produces wrong answers on specific rotations, not on all inputs — which is why they are easy to ship broken.
- 6 **Reading `a[mid + 1]` without proving `mid < hi`.** It is safe when `hi` starts at `n - 1` and the midpoint rounds down, but you should be able to say why rather than hope.
- 7 **Computing the midpoint as `(lo + hi) // 2`.** Python is safe, but every interviewer reads `lo + (hi - lo) // 2` as a signal that you know why the naive form is a bug in other languages. Write the safe form.
- 8 **Using `while lo < hi` when the target may be absent.** Shape B cannot report absence. If the problem needs a `-1`, either use Shape A or add an explicit validity check after the loop.

## Drill Set

Twenty-two problems, ordered so that each one adds exactly one idea to the previous. Do them without looking at the template. If a problem takes more than eight

minutes, stop, re-read the boundary rule it violates, and start again from the template.

Target: forty minutes for the first twelve, thirty for the rest. If you finish faster than that you are guessing rather than applying the rules, and the mistakes will surface under interview pressure.

| #    | PROBLEM                                   | LEVEL  | ASKED | WHAT CHANGES                                                                          |
|------|-------------------------------------------|--------|-------|---------------------------------------------------------------------------------------|
| 33   | Search in Rotated Sorted Array            | Medium | 21    |                                                                                       |
| 162  | Find Peak Element                         | Medium | 8     |                                                                                       |
| 278  | First Bad Version                         | Easy   | 7     |                                                                                       |
| 74   | Search a 2D Matrix                        | Medium | 10    | Flatten: <code>row, col = divmod(mid, n)</code> – total order makes it 1-D            |
| 1539 | Kth Missing Positive Number               | Easy   | 5     | Shape B: missing count at <code>mid</code> is <code>a[mid] - mid - 1 &gt;= k</code>   |
| 704  | Binary Search                             | Easy   | 9     |                                                                                       |
| 35   | Search Insert Position                    | Easy   | 9     |                                                                                       |
| 69   | Sqrt(x)                                   | Easy   | 7     |                                                                                       |
| 153  | Find Minimum in Rotated Sorted Array      | Medium | 7     |                                                                                       |
| 540  | Single Element in a Sorted Array          | Medium | 6     |                                                                                       |
| 240  | Search a 2D Matrix II                     | Medium | 6     | Not binary search – walk from the top-right corner, $O(m+n)$                          |
| 852  | Peak Index in a Mountain Array            | Medium | 4     |                                                                                       |
| 744  | Find Smallest Letter Greater Than Target  | Easy   | 3     | <code>bisect_right</code> ; wrap to <code>a[0]</code> when the result is past the end |
| 367  | Valid Perfect Square                      | Easy   | 2     |                                                                                       |
| 374  | Guess Number Higher or Lower              | Easy   | 2     |                                                                                       |
| 1351 | Count Negative Numbers in a Sorted Matrix | Easy   | 1     | <code>bisect_left</code> per row, or the 240 staircase walk                           |
| 275  | H-Index II                                | Medium | 1     | Shape B: first <code>i</code> with <code>citations[i] &gt;= n - i</code>              |

| #    | PROBLEM                                                 | LEVEL  | ASKED | WHAT CHANGES                                                           |
|------|---------------------------------------------------------|--------|-------|------------------------------------------------------------------------|
| 34   | Find First and Last Position of Element in Sorted Array | Medium | 0     |                                                                        |
| 441  | Arranging Coins                                         | Easy   | 0     | Shape B on <code>k(k+1)/2 &lt;= n</code> , or close the quadratic form |
| 1064 | Fixed Point †                                           | Easy   | 0     | Shape B: first <code>i</code> with <code>a[i] &gt;= i</code>           |
| 2300 | Successful Pairs of Spells and Potions                  | Medium | 0     | Sort potions, <code>bisect_left</code> per spell                       |
| 2529 | Maximum Count of Positive Integer and Negative Integer  | Easy   | 0     | <code>bisect_left(0)</code> and <code>n - bisect_right(0)</code>       |

PART II

# Sequential Scans

---

One-dimensional scans. Pointers and windows, and the boundary conditions that decide whether they are correct.

# Two Pointers - Converging

## THE KERNEL

One pointer at each end, both moving inward. Every problem here is the same `while l < r:` loop; only the move rule and the update rule differ.

This is the cheapest pattern in the book. There is one loop, two pointers, and a rule that says which one moves. Once the array is sorted, a pair is no longer something you search for — it is something you walk toward, and the walk is  $O(n)$  instead of  $O(n^2)$ .

The reason it works is a monotonicity argument, and you should be able to state it out loud in an interview. Put the smallest element on the left and the largest on the right. If their sum is too small, then the smallest element cannot be part of any valid pair — it is too small to pair with anything, including the largest value available. So discard it and move on. If the sum is too large, by the same argument the largest element cannot be in any valid pair. Every iteration removes one candidate permanently, which is what makes a single linear pass enough.

Everything else — 3Sum, 4Sum, closest pair, container with most water, palindrome checking — is that argument wearing a different costume. The chapter is short because there is genuinely only one idea in it.

**32**PROBLEMS  
MAPPED**14**

EASY

**17**

MEDIUM

**1**

HARD

## The Boundary Rules

---

Seven rules. Rule 1 and rule 2 are the loop itself; rule 3 is the precondition that makes the loop valid; rules 4 through 7 are the traps that separate a working answer from an accepted one.

- 1 Loop condition is always `while l < r` (never `<=`) — a converging pair must have two distinct indices.
- 2 Move exactly one pointer per iteration, decided by a comparison against the target. Moving both is the classic bug that silently skips candidates.
- 3 If the array is not sorted and the problem is about pairs/triplets, sort first — the  $O(n \log n)$  sort is what buys you the  $O(n)$  scan.
- 4 After sorting, the index you get back is an index into the *sorted* array. If the problem asks for original indices, you must carry them.
- 5 Dedup rule: skip a value only when it equals the previous *kept* value, and only while `l < r`. `if l > start and a[l] == a[l-1]: continue`.
- 6 3Sum is `for i in range(n-2)` wrapping a converging pair; the outer loop carries its own dedup and its own early exit `if a[i] > 0: break`.
- 7 Container-with-most-water never needs sorting: the greedy move is `move the shorter side`, because the shorter side is the bottleneck.

## The Template

---

There is one frame and one shell. The frame is the converging loop. The shell is the deduplication wrapper you need whenever the problem asks for a *set* of tuples rather than a single pair, which is where most submissions fail.

Memorise the frame. The shell you can rebuild each time from rule 5, as long as you remember that duplicates are skipped at the anchor and at the pointer separately — they are two different loops and they are both required.

```
# ----- the frame
# Sorted array, one pointer at each end, both moving inward.
```

```

def two_sum_sorted(a, target):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target:
            return [l, r]
        if s < target:
            l += 1
        else:
            r -= 1
    return []

# ----- the dedup shell
# Use whenever the answer is a SET of tuples, not one pair.

def three_sum(a):
    a.sort()
    out = []
    for i in range(len(a) - 2):
        if a[i] > 0:
            break
        if i > 0 and a[i] == a[i - 1]:
            continue
        l, r = i + 1, len(a) - 1
        while l < r:
            s = a[i] + a[l] + a[r]
            if s == 0:
                out.append([a[i], a[l], a[r]])
                l += 1
                while l < r and a[l] == a[l - 1]:
                    l += 1
            elif s < 0:
                l += 1
            else:
                r -= 1
    return out

```

## Why moving both pointers at once is wrong

It is tempting to think that when the sum is too small you should advance `l` and retreat `r`, since neither endpoint looked right. That is a bug, and it is a silent one: it skips candidates without ever failing a test you would notice on a small example.

The monotonicity argument is what tells you the truth. When `a[l] + a[r]` is too small, `a[l]` is too small to pair with `a[r]` — the largest remaining value. That tells you nothing about `a[r]`. `a[r]` may well be the correct partner for some *other* element further in. Moving `r` discards it on no evidence.

So the rule is mechanical: exactly one pointer moves per iteration, and the comparison decides which. If you find yourself writing both, you have lost the invariant and the linear bound with it.

## The two deduplication loops are not the same loop

In the `three_sum` shell there are two places duplicates are skipped, and they exist for different reasons. The outer check `if i > 0 and a[i] == a[i - 1]: continue` skips a repeated **anchor** — if you have already produced every triple starting with the value 2, starting again at another 2 produces the same set.

The inner loop after a hit skips repeated **partners** — once you have emitted `[-4, 2, 2]`, advancing `l` onto a second `2` would emit it again.

The trap is that the inner loop must be guarded by `l < r`, not by a fixed bound. Without it, a long run of equal values can walk `l` past `r` and produce a pair that reuses the same element twice, which is not a pair at all.

## Worked Solutions

Eight problems. Each one is the frame with one decision changed, and the decision is named in the heading so you can find it again under pressure.

### 167 Two Sum II - Input Array Is Sorted

LC 167 · Medium · asked by 6 companies · AC 65.9% · Array, Two Pointers, Binary Search

#### THE PROBLEM

You are given a **1-indexed** array of integers `numbers` that is already **sorted in non-decreasing order**.

Find **two** numbers such that they add up to a specific `target` number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where  $1 \leq \text{index}_1 < \text{index}_2 \leq \text{numbers.length}$ .

Return the indices of the two numbers `index1` and `index2` as an integer array `[index1, index2]` of length 2.

The tests are generated such that there is **exactly one solution**. You **may not** use the same element twice.

Your solution must use only constant extra space.

### Example 1:

**Input:** `numbers = [2,7,11,15]`, `target = 9`  
**Output:** `[1,2]`

**Explanation** The sum of 2 and 7 is 9. Therefore, `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

### Example 2:

**Input:** `numbers = [2,3,4]`, `target = 6`  
**Output:** `[1,3]`

**Explanation** The sum of 2 and 4 is 6. Therefore `index1 = 1`, `index2 = 3`. We return `[1, 3]`.

### Example 3:

**Input:** `numbers = [-1,0]`, `target = -1`  
**Output:** `[1,2]`

**Explanation** The sum of -1 and 0 is -1. Therefore `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

### Constraints:

- $2 \leq \text{numbers.length} \leq 3 * 10^4$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- `numbers` is sorted in **non-decreasing order**.
- $-1000 \leq \text{target} \leq 1000$
- The tests are generated such that there is **exactly one solution**.

**SIGNATURE**

```
def twoSum(numbers, target): # -> integer[]
```

**IN ONE LINE** In a sorted array, return the 1-indexed positions of the two numbers that add to `target`.

**THE INSIGHT** The frame in its purest form. The array is already sorted, so the only work is the comparison.

`l` starts at the smallest value and `r` at the largest. If the sum is too small, the smallest value cannot participate in any valid pair, so `l` advances. If too large, the largest value cannot, so `r` retreats. Each step discards one index for good.

The only thing the problem adds over the template is the off-by-one in the return: the statement is 1-indexed while Python is 0-indexed. That single `+ 1` on each end is the most commonly lost point on this problem.

```
def twoSum(numbers, target):
    l, r = 0, len(numbers) - 1
    while l < r:
        s = numbers[l] + numbers[r]
        if s == target:
            return [l + 1, r + 1] # the statement is 1-indexed
        if s < target:
            l += 1
        else:
            r -= 1
    return []
```

**Complexity.**  $O(n)$  time,  $O(1)$  space. Compare the hash-map solution for `Two Sum` (1):  $O(n)$  time but  $O(n)$  space. Sortedness is what lets you drop the memory.

**PITFALL**

Returning `[l, r]` instead of `[l + 1, r + 1]`.

**PITFALL**

Writing `while l <= r`. With `<=` the loop can settle on a single index and the function reports a number paired with itself.

**Follow-up.** “What if the array were not sorted?” Then either sort first –  $O(n \log n)$  – or use the hash map from chapter 20. The converging frame requires order.

## 125 Valid Palindrome

LC 125 · Easy · asked by 6 companies · AC 54.3% · Two Pointers, String

### THE PROBLEM

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` if it is a **palindrome**, or `false` otherwise.

#### Example 1:

**Input:** `s = "A man, a plan, a canal: Panama"`  
**Output:** `true`

**Explanation** “amanaplanacanalpanama” is a palindrome.

#### Example 2:

**Input:** `s = "race a car"`  
**Output:** `false`

**Explanation** “raceacar” is not a palindrome.

#### Example 3:

**Input:** `s = ""`  
**Output:** `true`

**Explanation** `s` is an empty string "" after removing non-alphanumeric characters. Since an empty string reads the same forward and backward, it is a palindrome.

#### Constraints:

- `1 <= s.length <= 2 * 105`
- `s` consists only of printable ASCII characters.

### SIGNATURE

```
def isPalindrome(s):    # -> boolean
```

**IN ONE LINE** Decide whether a string reads the same forwards and backwards, ignoring case and non-alphanumeric characters.

**THE INSIGHT** Same frame, but the pointers must skip characters that do not count, and the skip loops need the same `l < r` guard as the outer loop.

The frame compares `s[l]` with `s[r]` and moves both inward — note that this is one of the few places both pointers move, because a palindrome match consumes both ends at once.

The new ingredient is the inner skip loops. A character that is not alphanumeric is invisible to the problem, so each pointer walks over it. The subtlety is that these loops must be guarded by `l < r` themselves. Without the guard, a string like `".,"` walks `l` off the end of the string and the next comparison reads out of range.

Normalise with `.lower()` at the point of comparison rather than rebuilding the string. Filtering into a new string is also correct but allocates  $O(n)$  memory for no reason.

```
def isPalindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum():
            l += 1
        while l < r and not s[r].isalnum():
            r -= 1
        if s[l].lower() != s[r].lower():
            return False
        l += 1
        r -= 1
    return True
```

**Complexity.**  $O(n)$  time,  $O(1)$  space. Each character is visited at most once by each pointer.

#### PITFALL

Omitting `l < r` inside the skip loops. This is the single most common failure on this problem.

**PITFALL**

Using `s[l].isalpha()` when the statement says alphanumeric — digits count, so `"0P"` is not a palindrome but `"0P0"` is.

**Follow-up.** "What if you may delete at most one character?" That is problem 680 below, and it is a one-line change with a trap.

**15 3Sum**

LC 15 · **Medium** · asked by 19 companies · AC 39.9% · Array, Two Pointers, Sorting

**THE PROBLEM**

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

**Example 1:**

**Input:** `nums = [-1,0,1,2,-1,-4]`  
**Output:** `[[-1,-1,2], [-1,0,1]]`

**Explanation** `nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0`. `nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0`. `nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0`. The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`. Notice that the order of the output and the order of the triplets does not matter.

**Example 2:**

**Input:** `nums = [0,1,1]`  
**Output:** `[]`

**Explanation** The only possible triplet does not sum up to 0.

**Example 3:**

**Input:** `nums = [0,0,0]`  
**Output:** `[[0,0,0]]`

**Explanation** The only possible triplet sums up to 0.

**Constraints:**

- `3 <= nums.length <= 3000`

- `-105 <= nums[i] <= 105`

**SIGNATURE**

```
def threeSum(nums): # -> list<list<integer>>
```

**IN ONE LINE** Return every triple of distinct positions whose values sum to zero, with no repeated triples.

**THE INSIGHT** Sort, then wrap the converging frame in a loop over the anchor. The whole difficulty is the two deduplication loops, not the search.

Sorting does two things at once: it makes the inner converging scan valid, and it makes duplicates adjacent, which is what makes them skippable at all. Without the sort you cannot dedup in  $O(1)$  space per candidate.

The outer loop fixes `nums[i]` as the anchor and asks the inner frame for a pair summing to `-nums[i]`. Two early exits matter. `if nums[i] > 0: break` is valid because once the anchor is positive, every later anchor is larger and no triple can reach zero. The duplicate-anchor check `if i > 0 and nums[i] == nums[i - 1]: continue` is what stops the same triple being emitted from different starting positions.

After a hit, `l` must skip past equal values. This is the second dedup loop, and it is guarded by `l < r` so it cannot overrun.

```
def threeSum(nums):
    nums.sort()
    out = []
    for i in range(len(nums) - 2):
        if nums[i] > 0:
            break # sorted: no later triple reaches 0
        if i > 0 and nums[i] == nums[i - 1]:
            continue # duplicate anchor
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s == 0:
                out.append([nums[i], nums[l], nums[r]])
                l += 1
                while l < r and nums[l] == nums[l - 1]:
                    l += 1
```

```

        l += 1          # duplicate partner
    elif s < 0:
        l += 1
    else:
        r -= 1
return out

```

**Complexity.**  $O(n^2)$  time –  $O(n \log n)$  to sort plus  $O(n)$  for each of  $n$  anchors.  $O(1)$  extra space beyond the output.

#### PITFALL

Keeping only one of the two dedup checks. With only the anchor check, a triple like `[-1, 0, 1]` is emitted once per duplicate partner; with only the partner check, it is emitted once per duplicate anchor.

#### PITFALL

Using a `set` of tuples instead of the two loops. It works, but it hides whether you understand the invariant, and interviewers ask.

#### PITFALL

Dropping the `l < r` guard inside the partner skip loop, which lets `l` run past `r` and reuse an element twice.

**Follow-up.** “And 4Sum?” Add a second anchor loop around the same shell –  $O(n^3)$ . The pattern does not change; only the nesting does.

## 11 Container With Most Water

LC 11 · **Medium** · asked by 9 companies · AC 60.9% · Array, Two Pointers, Greedy

### THE PROBLEM

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the  $i^{\text{th}}$  line are `(i, 0)` and `(i, height[i])`.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

**Notice** that you may not slant the container.

**Example 1:**

[diagram omitted — see [leetcode.com/problems/container-with-most-water/](https://leetcode.com/problems/container-with-most-water/)]

**Input:** height = [1,8,6,2,5,4,8,3,7]  
**Output:** 49

**Explanation** The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

**Example 2:**

**Input:** height = [1,1]  
**Output:** 1

**Constraints:**

- `n == height.length`
- `2 <= n <= 105`
- `0 <= height[i] <= 104`

**SIGNATURE**

```
def maxArea(height): # -> integer
```

**IN ONE LINE** Given vertical lines at each index, choose two that with the x-axis hold the most water.

**THE INSIGHT** The only problem in the chapter that must **not** be sorted, because the answer depends on the original positions. The move rule is different too: move the shorter side.

The area is `min(height[l], height[r]) * (r - l)`. Sorting would destroy the widths, so the array stays as it is and the frame runs on the original indices.

The move rule follows from the bottleneck argument. The shorter wall caps the area. Moving the taller wall inward can only reduce the width while the cap stays the same, so it can never improve. Moving the shorter wall is the only move that has any chance of finding a larger cap. This is the same “discard the endpoint that

cannot participate” argument as the sorted case, applied to a different monotonicity.

Note what you do **not** do: you never look ahead. The greedy move is locally justified and globally optimal here, and the proof is the bottleneck argument above.

```
def maxArea(height):
    l, r = 0, len(height) - 1
    best = 0
    while l < r:
        h = min(height[l], height[r])
        best = max(best, h * (r - l))
        if height[l] < height[r]:
            l += 1          # the shorter wall is the bottleneck
        else:
            r -= 1
    return best
```

**Complexity.**  $O(n)$  time,  $O(1)$  space. One comparison and one move per iteration.

#### PITFALL

Sorting the array first. The area depends on index distance, so sorting changes the answer — this is the trap that catches people who have just learned 3Sum.

#### PITFALL

Moving the taller wall. It looks harmless and is wrong on inputs where the optimum needs both walls from one half.

#### PITFALL

Computing  $h * (r - l)$  with  $h = \text{height}[l]$  only, instead of the minimum of the two.

**Follow-up.** “Prove the greedy move.” This is the standard exchange argument: any solution that keeps the shorter wall and discards the taller one is dominated by the move the algorithm makes.

**16 3Sum Closest**LC 16 · **Medium** · asked by 7 companies · AC 47.4% · Array, Two Pointers, Sorting**THE PROBLEM**

You are given an integer array `nums` of length `n` and an integer `target`.

Find three integers at **distinct indices** in `nums` such that the sum is **closest** to `target`.

Return the sum of the three integers.

You may assume that each input would have **exactly** one solution.

**Example 1:**

**Input:** `nums = [-1,2,1,-4]`, `target = 1`  
**Output:** `2`

**Explanation** The sum that is closest to the target is 2.  $(-1 + 2 + 1 = 2)$ .

**Example 2:**

**Input:** `nums = [0,0,0]`, `target = 1`  
**Output:** `0`

**Explanation** The sum that is closest to the target is 0.  $(0 + 0 + 0 = 0)$ .

**Constraints:**

- `3 <= nums.length <= 500`
- `-1000 <= nums[i] <= 1000`
- `-104 <= target <= 104`

**SIGNATURE**

```
def threeSumClosest(nums, target): # -> integer
```

**IN ONE LINE** Find the triple whose sum is closest to a target.

**THE INSIGHT** The 3Sum shell with the equality test replaced by a running best. No deduplication is needed, because the answer is a single number rather than a set of tuples.

This is the clearest demonstration that the two dedup loops in 3Sum are not part of the frame — they belong to the *output shape*. Here the output is one integer, so both loops vanish and the code gets shorter.

The frame is unchanged: too small advances `l`, too large retreats `r`. The only addition is that every candidate is compared against the best so far, since the exact target may never be hit.

The early return when `s == target` is worth keeping: an exact hit is optimal and no further work can improve it.

```
def threeSumClosest(nums, target):
    nums.sort()
    best = nums[0] + nums[1] + nums[2]
    for i in range(len(nums) - 2):
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if abs(s - target) < abs(best - target):
                best = s
            if s < target:
                l += 1
            elif s > target:
                r -= 1
            else:
                return target          # exact hit: cannot be beaten
    return best
```

**Complexity.**  $O(n^2)$  time,  $O(1)$  space. The early return makes the common case much faster than the bound suggests.

#### PITFALL

Seeding `best` with a sentinel like `float('inf')` and then returning it when the array is short. Seed from the first triple instead — the array always has at least three elements.

#### PITFALL

Forgetting the exact-hit early return, which is not a correctness bug but is the difference between passing and looking like you know the problem.

**Follow-up.** “Closest pair in a sorted array?” Same running-best idea with the plain frame, no anchor loop.

**42 Trapping Rain Water**

LC 42 · **Hard** · asked by 18 companies · AC 68.2% · Array, Two Pointers, Dynamic Programming, Stack

**THE PROBLEM**

Given `n` non-negative integers representing an elevation map where the width of each bar is `1`, compute how much water it can trap after raining.

**Example 1:**

[diagram omitted — see [leetcode.com/problems/trapping-rain-water/](https://leetcode.com/problems/trapping-rain-water/)]

**Input:** height = [0,1,0,2,1,0,1,3,2,1,2,1]  
**Output:** 6

**Explanation** The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

**Example 2:**

**Input:** height = [4,2,0,3,2,5]  
**Output:** 9

**Constraints:**

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`

**SIGNATURE**

```
def trap(height):    # -> integer
```

**IN ONE LINE** Given an elevation map, compute how much rain it traps.

**THE INSIGHT** The hardest problem in the chapter, and it is still the same frame. The water above any bar is `min(left_max, right_max) - height`, and the converging scan can supply that minimum without ever building either array.

The straightforward solution computes two prefix arrays — the tallest bar to the left of each index and the tallest to the right — then sums `min(left, right) - height`. That is  $O(n)$  time and  $O(n)$  space, and it is a perfectly good answer.

The two-pointer version removes the arrays. The key observation: when `height[l] < height[r]`, the water above `l` is determined *entirely* by `left_max`, because there is a bar at `r` at least as tall as `height[l]` somewhere to the right, which is enough to guarantee `right_max >= height[l]`. So `min(left_max, right_max) == left_max` and you can settle `l` immediately without knowing `right_max` at all.

That is the whole trick, and it is worth stating precisely: you never need both maxima, only the one on the side of the shorter bar. The loop advances whichever side is shorter and settles that index permanently.

```
def trap(height):
    if not height:
        return 0
    l, r = 0, len(height) - 1
    left_max = right_max = 0
    water = 0
    while l < r:
        if height[l] < height[r]:
            left_max = max(left_max, height[l])
            water += left_max - height[l]
            l += 1
        else:
            right_max = max(right_max, height[r])
            water += right_max - height[r]
            r -= 1
    return water
```

**Complexity.**  $O(n)$  time,  $O(1)$  space. Strictly better than the prefix-array solution on memory, and the version an interviewer is hoping you reach.

#### PITFALL

Comparing `height[l]` with `height[r]` and then using the wrong maximum. The rule is: whichever bar is shorter, its side's maximum is the binding constraint. Swap them and the answer is wrong on asymmetric inputs.

**PITFALL**

Updating `left_max` after accumulating instead of before, which adds a negative term at every new maximum.

**PITFALL**

Forgetting the empty-input guard, which makes `l` and `r` invalid on an empty array.

**Follow-up.** “What about the 2-D version?” Problem 407. It needs a heap rather than two pointers – see chapter 32 – because there is no single pair of maxima that bounds the water.

## 680 Valid Palindrome II

LC 680 · **Easy** · asked by 7 companies · AC 44.8% · Two Pointers, String, Greedy

**THE PROBLEM**

Given a string `s`, return `true` if the `s` can be palindrome after deleting **at most one** character from it.

**Example 1:**

**Input:** `s = "aba"`  
**Output:** `true`

**Example 2:**

**Input:** `s = "abca"`  
**Output:** `true`

**Explanation** You could delete the character ‘c’.

**Example 3:**

**Input:** `s = "abc"`  
**Output:** `false`

**Constraints:**

- `1 <= s.length <= 105`
- `s` consists of lowercase English letters.

**SIGNATURE**

```
def validPalindrome(s):    # -> boolean
```

**IN ONE LINE** Decide whether a string is a palindrome after deleting at most one character.

**THE INSIGHT** Run the frame; at the first mismatch, the answer is the OR of the two possible deletions. Only one branch can be right, and you must try both.

The frame advances while the characters match. The moment they differ, exactly one of two things must be true: the left character is the extra one, or the right character is. So you check the remaining substring both ways and accept if either works.

The trap is thinking you can decide *which* one to delete from the local comparison. You cannot. Given "abca" mismatching at the outer pair, both branches must be tested, because the surrounding context decides it and you have not looked at the surroundings yet.

Factor the plain check into a helper. The outer loop runs once and calls the helper at most twice, which is why the total work is still linear — each helper call scans a substring, and there are only two of them.

```
def validPalindrome(s):
    def ok(i, j):
        while i < j:
            if s[i] != s[j]:
                return False
            i += 1
            j -= 1
        return True

    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]:
            return ok(l + 1, r) or ok(l, r - 1)
        l += 1
        r -= 1
    return True
```

**Complexity.**  $O(n)$  time – the outer loop plus at most two linear scans.  $O(1)$  space beyond the recursion-free helper.

**PITFALL**

Trying only one branch, e.g. always skipping `s[l]`. Which character is the extra one depends on the rest of the string, not on the mismatch itself.

**PITFALL**

Recurring instead of using a helper and accidentally making it exponential, because each mismatch can spawn two more.

**PITFALL**

Returning `False` after both branches fail but forgetting that the loop must also return `True` when no mismatch is found at all.

**Follow-up.** "At most k deletions?" That is the two-sequence DP of chapter 62 – the greedy two-branch trick works only for  $k = 1$ .

## 977 Squares of a Sorted Array

LC 977 · **Easy** · asked by 5 companies · AC 74.2% · Array, Two Pointers, Sorting

**THE PROBLEM**

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of the squares of each number sorted in non-decreasing order*.

**Example 1:**

**Input:** `nums = [-4,-1,0,3,10]`  
**Output:** `[0,1,9,16,100]`

**Explanation** After squaring, the array becomes `[16,1,0,9,100]`. After sorting, it becomes `[0,1,9,16,100]`.

**Example 2:**

**Input:** `nums = [-7,-3,2,3,11]`  
**Output:** `[4,9,9,49,121]`

**Constraints:**

- `1 <= nums.length <= 104`
- `-104 <= nums[i] <= 104`

- `nums` is sorted in **non-decreasing** order.

**SIGNATURE**

```
def sortedSquares(nums): # -> integer[]
```

**IN ONE LINE** Given a sorted array, return the squares of its elements in sorted order.

**THE INSIGHT** The largest square is at one of the two ends, so fill the output from the back while converging inward.

The array is sorted, but its *squares* are not: a large negative number at the left can square to more than a large positive at the right. So the usual “smallest first” order is broken.

The fix is to reverse the direction of the fill. The largest square is always at one of the two ends, because that is where the largest magnitudes live. Compare `abs(nums[l])` with `abs(nums[r])`, take the bigger square, place it at the current back position, and advance that pointer. The output fills from the last index down to zero.

Note that both pointers move inward but only one per iteration, exactly as rule 2 requires. Writing into a preallocated list from the back avoids the `insert(0, ...)` that would make the solution quadratic.

```
def sortedSquares(nums):
    n = len(nums)
    out = [0] * n
    l, r = 0, n - 1
    for k in range(n - 1, -1, -1):
        if abs(nums[l]) > abs(nums[r]):
            out[k] = nums[l] * nums[l]
            l += 1
        else:
            out[k] = nums[r] * nums[r]
            r -= 1
    return out
```

**Complexity.**  $O(n)$  time,  $O(n)$  space for the output. Beats the obvious sort-the-squares approach, which is  $O(n \log n)$ .

**PITFALL**

Using `nums[l] > nums[r]` without `abs`. The squares of negatives compare wrongly if you forget.

**PITFALL**

Writing to the front with `out.append(...)` and then reversing, which is correct but obscures the direction of the fill.

**PITFALL**

Using `insert(0, x)`, which is  $O(n)$  per call and quietly turns a linear solution into a quadratic one.

**Follow-up.** “Why is the maximum always at an end?” Because  $x^2$  is monotone in  $|x|$ , and the extremes of  $|x|$  are at the ends of a sorted array.

## The Index – What Changes in the Template

Every remaining problem in this chapter, sorted by how often it is asked. The last column names the single thing that changes in the frame — read that column and nothing else on a first pass.

| #   | PROBLEM                    | LEVEL  | ASKED | WHAT CHANGES                                                                         |
|-----|----------------------------|--------|-------|--------------------------------------------------------------------------------------|
| 349 | Intersection of Two Arrays | Easy   | 7     | Sort both, then converge on equality; advance the side that is smaller               |
| 18  | 4Sum                       | Medium | 8     | Two anchor loops around the ch 5 frame; the dedup rules apply at <b>every</b> level  |
| 344 | Reverse String             | Easy   | 6     | The frame with both pointers always moving – one <code>swap</code> per iteration     |
| 844 | Backspace String Compare   | Easy   | 7     | Converge from the <b>back</b> , skipping characters cancelled by <code>#</code>      |
| 881 | Boats to Save People       | Medium | 7     | Sort; heaviest with lightest – both board if they fit, otherwise the heavy one alone |
| 532 | K-diff Pairs in an Array   | Medium | 7     | Sort then converge, or hash the counts; dedup on the <b>value</b> , not the index    |

## TWO POINTERS - CONVERGING

| #    | PROBLEM                                                     | LEVEL  | ASKED | WHAT CHANGES                                                                                                                          |
|------|-------------------------------------------------------------|--------|-------|---------------------------------------------------------------------------------------------------------------------------------------|
| 392  | Is Subsequence                                              | Easy   | 3     | Two pointers moving the <b>same</b> direction, not converging – chapter 6                                                             |
| 1099 | Two Sum Less Than K †                                       | Easy   | 2     | The frame with <code>&lt;</code> instead of <code>==</code> ; track the best valid sum                                                |
| 611  | Valid Triangle Number                                       | Medium | 3     | Sort; fix the largest side as <code>r</code> , then count pairs with <code>a[l] + a[mid] &gt; a[r]</code>                             |
| 923  | 3Sum With Multiplicity                                      | Medium | 2     | Same frame; count combinations instead of emitting, over <code>i &lt; j &lt; k</code>                                                 |
| 345  | Reverse Vowels of a String                                  | Easy   | 1     | The frame plus skip loops, like 125, with a swap on a match                                                                           |
| 1471 | The k Strongest Values in an Array                          | Medium | 1     | Sort by distance from the median, which is <code>a[(n - 1) // 2]</code>                                                               |
| 2149 | Rearrange Array Elements by Sign                            | Medium | 1     | Fill a preallocated output from two write pointers, one per parity                                                                    |
| 259  | 3Sum Smaller †                                              | Medium | 0     | Sorted + converging: when <code>s &lt; target</code> , all <code>r - l</code> pairs count at once – add them and move <code>l</code>  |
| 905  | Sort Array By Parity                                        | Easy   | 0     | Converging partition: swap when <code>l</code> is odd and <code>r</code> is even                                                      |
| 948  | Bag of Tokens                                               | Medium | 0     | Sort; spend tokens at the cheap end, gain at the expensive end                                                                        |
| 1498 | Number of Subsequences That Satisfy the Given Sum Condition | Medium | 0     | Sort; for each <code>l</code> find the largest <code>r</code> with <code>a[l] + a[r] &lt;= t</code> , then add <code>2^(r-l-1)</code> |
| 2105 | Watering Plants II                                          | Medium | 0     | Two pointers over the <b>plants</b> , one from each side – not the array ends                                                         |
| 2330 | Valid Palindrome IV †                                       | Medium | 0     | Count mismatched pairs in the frame; at most two may differ                                                                           |
| 2441 | Largest Positive Integer That Exists With Its Negative      | Easy   | 0     | A hash set, not two pointers – there is no ordering to exploit                                                                        |
| 2562 | Find the Array Concatenation Value                          | Easy   | 0     | The frame; concatenate <code>str(a[l]) + str(a[r])</code> and accumulate                                                              |
| 2824 | Count Pairs Whose Sum is Less than Target                   | Easy   | 0     | <code>n &lt;= 50</code> so brute force passes, but sorted + converging counts in one pass                                             |

| #    | PROBLEM                                           | LEVEL  | ASKED | WHAT CHANGES                                                                             |
|------|---------------------------------------------------|--------|-------|------------------------------------------------------------------------------------------|
| 2825 | Make String a Subsequence Using Cyclic Increments | Medium | 0     | Two pointers with a cyclic-increment budget carried on each character                    |
| 2932 | Maximum Strong Pair XOR I                         | Easy   | 0     | <code>n &lt;= 50</code> : brute force. The fast version needs a binary trie – chapter 90 |

## Pitfalls

- Sorting a problem that depends on original positions.** Rule 3 tells you to sort for pair problems. It does not apply to `Container With Most Water` (11), `Trapping Rain Water` (42) or `Boats to Save People` (881) — in the first two the answer is the distance between indices, and sorting destroys it. Ask whether the answer depends on positions before you call `.sort()`.
- Moving both pointers in one iteration.** The comparison justifies discarding exactly one endpoint. Moving both discards a candidate on no evidence and silently skips valid answers.
- Writing `while l <= r`.** A pair needs two distinct indices. With `<=`, the loop can settle on `l == r` and pair an element with itself.
- Keeping only one of the two dedup loops.** In 3Sum they are separate for a reason: one skips duplicate anchors, the other skips duplicate partners. Removing either produces repeated triples.
- The partner skip loop without its own `l < r` guard.** A long run of equal values walks `l` past `r`, and the next comparison reads the same element twice or goes out of range.
- Forgetting that sorting costs  $O(n \log n)$ .** The frame is  $O(n)$ , but the problem is not linear unless the input arrives sorted. Say the bound as  $O(n \log n)$  when you sort.
- Reaching for two pointers when `n` is tiny.** `Maximum Strong Pair XOR` (2932) has `n <= 50`; brute force is the intended answer and the converging frame does not apply at all.
- Assuming the frame needs a `for` loop.** The frame is a `while l < r`. If you find yourself indexing with `for i in range(...)`, you are probably in a different pattern.

## Drill Set

Thirty problems, ordered so that each adds exactly one idea to the previous. The first eight are the frame alone; 15 and 16 introduce the dedup shell; 42 is the only one that needs an argument rather than a substitution.

Do them without looking at the template. If a problem takes more than eight minutes, stop, re-read the boundary rule it violates, and restart from the frame.

Target: thirty minutes for the first twelve, forty for the rest. If you finish faster you are guessing rather than applying the rules, and the mistakes will surface under interview pressure.

| #   | PROBLEM                            | LEVEL  | ASKED | WHAT CHANGES                                                                         |
|-----|------------------------------------|--------|-------|--------------------------------------------------------------------------------------|
| 680 | Valid Palindrome II                | Easy   | 7     |                                                                                      |
| 42  | Trapping Rain Water                | Hard   | 18    |                                                                                      |
| 15  | 3Sum                               | Medium | 19    |                                                                                      |
| 125 | Valid Palindrome                   | Easy   | 6     |                                                                                      |
| 11  | Container With Most Water          | Medium | 9     |                                                                                      |
| 977 | Squares of a Sorted Array          | Easy   | 5     |                                                                                      |
| 16  | 3Sum Closest                       | Medium | 7     |                                                                                      |
| 349 | Intersection of Two Arrays         | Easy   | 7     | Sort both, then converge on equality; advance the side that is smaller               |
| 18  | 4Sum                               | Medium | 8     | Two anchor loops around the ch 5 frame; the dedup rules apply at <b>every</b> level  |
| 167 | Two Sum II - Input Array Is Sorted | Medium | 6     |                                                                                      |
| 344 | Reverse String                     | Easy   | 6     | The frame with both pointers always moving – one <code>swap</code> per iteration     |
| 844 | Backspace String Compare           | Easy   | 7     | Converge from the <b>back</b> , skipping characters cancelled by <code>#</code>      |
| 881 | Boats to Save People               | Medium | 7     | Sort; heaviest with lightest – both board if they fit, otherwise the heavy one alone |
| 532 | K-diff Pairs in an Array           | Medium | 7     | Sort then converge, or hash the counts; dedup on the <b>value</b> , not the index    |

| #    | PROBLEM                                                     | LEVEL  | ASKED | WHAT CHANGES                                                                                                            |
|------|-------------------------------------------------------------|--------|-------|-------------------------------------------------------------------------------------------------------------------------|
| 392  | Is Subsequence                                              | Easy   | 3     | Two pointers moving the <b>same</b> direction, not converging – chapter 6                                               |
| 1099 | Two Sum Less Than K †                                       | Easy   | 2     | The frame with < instead of ==; track the best valid sum                                                                |
| 611  | Valid Triangle Number                                       | Medium | 3     | Sort; fix the largest side as r, then count pairs with <code>a[l] + a[mid] &gt; a[r]</code>                             |
| 923  | 3Sum With Multiplicity                                      | Medium | 2     | Same frame; count combinations instead of emitting, over <code>i &lt; j &lt; k</code>                                   |
| 345  | Reverse Vowels of a String                                  | Easy   | 1     | The frame plus skip loops, like 125, with a swap on a match                                                             |
| 1471 | The k Strongest Values in an Array                          | Medium | 1     | Sort by distance from the median, which is <code>a[(n - 1) // 2]</code>                                                 |
| 2149 | Rearrange Array Elements by Sign                            | Medium | 1     | Fill a preallocated output from two write pointers, one per parity                                                      |
| 259  | 3Sum Smaller †                                              | Medium | 0     | Sorted + converging: when <code>s &lt; target</code> , all <code>r - l</code> pairs count at once – add them and move l |
| 905  | Sort Array By Parity                                        | Easy   | 0     | Converging partition: swap when l is odd and r is even                                                                  |
| 948  | Bag of Tokens                                               | Medium | 0     | Sort; spend tokens at the cheap end, gain at the expensive end                                                          |
| 1498 | Number of Subsequences That Satisfy the Given Sum Condition | Medium | 0     | Sort; for each l find the largest r with <code>a[l] + a[r] &lt;= t</code> , then add <code>2^(r-l-1)</code>             |
| 2105 | Watering Plants II                                          | Medium | 0     | Two pointers over the <b>plants</b> , one from each side – not the array ends                                           |
| 2330 | Valid Palindrome IV †                                       | Medium | 0     | Count mismatched pairs in the frame; at most two may differ                                                             |
| 2441 | Largest Positive Integer That Exists With Its Negative      | Easy   | 0     | A hash set, not two pointers – there is no ordering to exploit                                                          |
| 2562 | Find the Array Concatenation Value                          | Easy   | 0     | The frame; concatenate <code>str(a[l]) + str(a[r])</code> and accumulate                                                |
| 2824 | Count Pairs Whose Sum is Less than Target                   | Easy   | 0     | <code>n &lt;= 50</code> so brute force passes, but sorted + converging counts in one pass                               |

PART XVII

# Appendices

---

The generated complete index: every problem in the dataset, with its difficulty, frequency score and assigned chapter.

# The Complete High-Frequency Index

## THE KERNEL

All 1151 problems that appear in at least one of the 70 company interview sets, sorted by weighted frequency. The Chapter column is the chapter where the problem is discussed; problems with no chapter number are the long tail of easy warm-ups.

The **Asked** column counts how many of the 70 company sets contain the problem, and **Score** weights tier-one companies triple. A dagger (†) marks LeetCode Premium.

Use this as a checklist rather than a reading list. The question to ask of each row is not *have I solved this* but *can I write its chapter's template from memory without looking*.

| #    | PROBLEM                                  | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------|--------|-------|-------|---------|
| 1    | Two Sum                                  | Easy   | 24    | 1081  | 20      |
| 56   | Merge Intervals                          | Medium | 23    | 1026  | 18      |
| 146  | LRU Cache                                | Medium | 23    | 1002  | 24      |
| 1249 | Minimum Remove to Make Valid Parentheses | Medium | 7     | 914   | 25      |
| 200  | Number of Islands                        | Medium | 24    | 912   | 49      |
| 680  | Valid Palindrome II                      | Easy   | 7     | 803   | 5       |
| 236  | Lowest Common Ancestor of a Binary Tree  | Medium | 7     | 703   | 44      |
| 560  | Subarray Sum Equals K                    | Medium | 15    | 639   | 15      |
| 314  | Binary Tree Vertical Order Traversal †   | Medium | 4     | 609   | 40      |
| 50   | Pow(x, n)                                | Medium | 9     | 608   | 80      |

| #    | PROBLEM                                        | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------|--------|-------|-------|---------|
| 215  | Kth Largest Element in an Array                | Medium | 9     | 594   | 30      |
| 528  | Random Pick with Weight                        | Medium | 8     | 592   | 87      |
| 3    | Longest Substring Without Repeating Characters | Medium | 18    | 559   | 8       |
| 973  | K Closest Points to Origin                     | Medium | 5     | 559   | 30      |
| 20   | Valid Parentheses                              | Easy   | 21    | 546   | 25      |
| 1650 | Lowest Common Ancestor of a Binary Tree III †  | Medium | 6     | 515   | 23      |
| 42   | Trapping Rain Water                            | Hard   | 18    | 503   | 5       |
| 347  | Top K Frequent Elements                        | Medium | 10    | 503   | 30      |
| 23   | Merge k Sorted Lists                           | Hard   | 14    | 489   | 30      |
| 121  | Best Time to Buy and Sell Stock                | Easy   | 24    | 487   | 61      |
| 53   | Maximum Subarray                               | Medium | 20    | 480   | 15      |
| 227  | Basic Calculator II                            | Medium | 10    | 468   | 25      |
| 938  | Range Sum of BST                               | Easy   | 3     | 468   | 38      |
| 15   | 3Sum                                           | Medium | 19    | 461   | 5       |
| 253  | Meeting Rooms II †                             | Medium | 9     | 451   | 18      |
| 33   | Search in Rotated Sorted Array                 | Medium | 21    | 446   | 10      |
| 49   | Group Anagrams                                 | Medium | 19    | 423   | 21      |
| 2    | Add Two Numbers                                | Medium | 12    | 407   | 36      |
| 1570 | Dot Product of Two Sparse Vectors †            | Medium | 5     | 393   | 88      |
| 199  | Binary Tree Right Side View                    | Medium | 8     | 392   | 40      |
| 71   | Simplify Path                                  | Medium | 4     | 384   | 29      |
| 339  | Nested List Weight Sum †                       | Medium | 3     | 380   | 51      |
| 4    | Median of Two Sorted Arrays                    | Hard   | 13    | 379   | 10      |
| 88   | Merge Sorted Array                             | Easy   | 9     | 374   | 6       |
| 394  | Decode String                                  | Medium | 15    | 361   | 25      |
| 125  | Valid Palindrome                               | Easy   | 6     | 360   | 5       |
| 408  | Valid Word Abbreviation †                      | Easy   | 2     | 351   | 82      |

| #    | PROBLEM                                   | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-------------------------------------------|--------|-------|-------|---------|
| 543  | Diameter of Binary Tree                   | Easy   | 6     | 339   | 41      |
| 79   | Word Search                               | Medium | 7     | 328   | 49      |
| 162  | Find Peak Element                         | Medium | 8     | 325   | 10      |
| 921  | Minimum Add to Make Parentheses Valid     | Medium | 4     | 322   | 25      |
| 987  | Vertical Order Traversal of a Binary Tree | Hard   | 5     | 319   | 40      |
| 138  | Copy List with Random Pointer             | Medium | 6     | 309   | 36      |
| 1091 | Shortest Path in Binary Matrix            | Medium | 6     | 304   | 52      |
| 5    | Longest Palindromic Substring             | Medium | 9     | 303   | 60      |
| 380  | Insert Delete GetRandom O(1)              | Medium | 8     | 291   | 24      |
| 212  | Word Search II                            | Hard   | 9     | 280   | 49      |
| 139  | Word Break                                | Medium | 8     | 279   | 65      |
| 13   | Roman to Integer                          | Easy   | 9     | 277   | 80      |
| 238  | Product of Array Except Self              | Medium | 8     | 275   | 15      |
| 366  | Find Leaves of Binary Tree †              | Medium | 5     | 275   | 41      |
| 150  | Evaluate Reverse Polish Notation          | Medium | 6     | 269   | 25      |
| 17   | Letter Combinations of a Phone Number     | Medium | 8     | 261   | 45      |
| 210  | Course Schedule II                        | Medium | 12    | 252   | 54      |
| 14   | Longest Common Prefix                     | Easy   | 9     | 249   | 82      |
| 239  | Sliding Window Maximum                    | Hard   | 17    | 247   | 28      |
| 1268 | Search Suggestions System                 | Medium | 6     | 238   | 89      |
| 124  | Binary Tree Maximum Path Sum              | Hard   | 12    | 236   | 41      |
| 415  | Add Strings                               | Easy   | 7     | 236   | 80      |
| 127  | Word Ladder                               | Hard   | 8     | 233   | 52      |
| 8    | String to Integer (atoi)                  | Medium | 8     | 224   | 81      |
| 78   | Subsets                                   | Medium | 10    | 224   | 45      |
| 103  | Binary Tree Zigzag Level Order Traversal  | Medium | 9     | 219   | 40      |
| 22   | Generate Parentheses                      | Medium | 12    | 216   | 25      |
| 297  | Serialize and Deserialize Binary Tree     | Hard   | 9     | 216   | 44      |

| #    | PROBLEM                              | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------|--------|-------|-------|---------|
| 54   | Spiral Matrix                        | Medium | 7     | 214   | 58      |
| 523  | Continuous Subarray Sum              | Medium | 7     | 214   | 15      |
| 173  | Binary Search Tree Iterator          | Medium | 6     | 213   | 39      |
| 791  | Custom Sort String                   | Medium | 2     | 207   | 21      |
| 387  | First Unique Character in a String   | Easy   | 8     | 201   | 21      |
| 207  | Course Schedule                      | Medium | 9     | 200   | 54      |
| 21   | Merge Two Sorted Lists               | Easy   | 9     | 198   | 36      |
| 1396 | Design Underground System            | Medium | 1     | 192   | 88      |
| 11   | Container With Most Water            | Medium | 9     | 191   | 5       |
| 695  | Max Area of Island                   | Medium | 8     | 191   | 51      |
| 76   | Minimum Window Substring             | Hard   | 10    | 190   | 8       |
| 133  | Clone Graph                          | Medium | 6     | 187   | 56      |
| 346  | Moving Average from Data Stream †    | Easy   | 4     | 185   | 24      |
| 234  | Palindrome Linked List               | Easy   | 11    | 182   | 34      |
| 98   | Validate Binary Search Tree          | Medium | 8     | 181   | 39      |
| 65   | Valid Number                         | Hard   | 2     | 179   | 82      |
| 48   | Rotate Image                         | Medium | 9     | 178   | 58      |
| 10   | Regular Expression Matching          | Hard   | 9     | 172   | 63      |
| 62   | Unique Paths                         | Medium | 7     | 170   | 67      |
| 283  | Move Zeroes                          | Easy   | 10    | 169   | 6       |
| 273  | Integer to English Words             | Hard   | 8     | 168   | 81      |
| 498  | Diagonal Traverse                    | Medium | 3     | 168   | 58      |
| 140  | Word Break II                        | Hard   | 8     | 164   | 48      |
| 269  | Alien Dictionary †                   | Hard   | 9     | 162   | 52      |
| 1152 | Analyze User Website Visit Pattern † | Medium | 2     | 162   | 82      |
| 39   | Combination Sum                      | Medium | 9     | 160   | 45      |
| 692  | Top K Frequent Words                 | Medium | 9     | 160   | 21      |
| 977  | Squares of a Sorted Array            | Easy   | 5     | 160   | 5       |

| #    | PROBLEM                                     | LEVEL  | ASKED | SCORE | CHAPTER |
|------|---------------------------------------------|--------|-------|-------|---------|
| 16   | 3Sum Closest                                | Medium | 7     | 159   | 5       |
| 41   | First Missing Positive                      | Hard   | 7     | 159   | 23      |
| 198  | House Robber                                | Medium | 9     | 159   | 59      |
| 278  | First Bad Version                           | Easy   | 7     | 159   | 10      |
| 19   | Remove Nth Node From End of List            | Medium | 8     | 157   | 35      |
| 26   | Remove Duplicates from Sorted Array         | Easy   | 9     | 157   | 6       |
| 68   | Text Justification                          | Hard   | 7     | 153   | 82      |
| 994  | Rotting Oranges                             | Medium | 4     | 153   | 52      |
| 670  | Maximum Swap                                | Medium | 4     | 150   | 81      |
| 1209 | Remove All Adjacent Duplicates in String II | Medium | 8     | 150   | 29      |
| 134  | Gas Station                                 | Medium | 9     | 149   | 76      |
| 74   | Search a 2D Matrix                          | Medium | 10    | 148   | 10      |
| 249  | Group Shifted Strings †                     | Medium | 2     | 147   | 22      |
| 438  | Find All Anagrams in a String               | Medium | 10    | 146   | 7       |
| 721  | Accounts Merge                              | Medium | 7     | 143   | 55      |
| 244  | Shortest Word Distance II †                 | Medium | 1     | 142   | 88      |
| 224  | Basic Calculator                            | Hard   | 9     | 141   | 25      |
| 155  | Min Stack                                   | Medium | 8     | 140   | 29      |
| 217  | Contains Duplicate                          | Easy   | 8     | 140   | 20      |
| 399  | Evaluate Division                           | Medium | 8     | 140   | 56      |
| 152  | Maximum Product Subarray                    | Medium | 6     | 137   | 15      |
| 322  | Coin Change                                 | Medium | 7     | 137   | 59      |
| 636  | Exclusive Time of Functions                 | Medium | 5     | 137   | 25      |
| 12   | Integer to Roman                            | Medium | 7     | 136   | 80      |
| 1822 | Sign of the Product of an Array             | Easy   | 1     | 135   | 80      |
| 735  | Asteroid Collision                          | Medium | 8     | 133   | 29      |
| 208  | Implement Trie (Prefix Tree)                | Medium | 9     | 132   | 89      |
| 1047 | Remove All Adjacent Duplicates In String    | Easy   | 2     | 132   | 29      |

| #    | PROBLEM                                 | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-----------------------------------------|--------|-------|-------|---------|
| 91   | Decode Ways                             | Medium | 9     | 131   | 60      |
| 151  | Reverse Words in a String               | Medium | 6     | 131   | 82      |
| 102  | Binary Tree Level Order Traversal       | Medium | 9     | 128   | 40      |
| 202  | Happy Number                            | Easy   | 6     | 127   | 6       |
| 348  | Design Tic-Tac-Toe †                    | Medium | 5     | 127   | 88      |
| 7    | Reverse Integer                         | Medium | 11    | 126   | 80      |
| 206  | Reverse Linked List                     | Easy   | 8     | 125   | 34      |
| 359  | Logger Rate Limiter †                   | Easy   | 6     | 125   | 24      |
| 242  | Valid Anagram                           | Easy   | 8     | 124   | 21      |
| 286  | Walls and Gates †                       | Medium | 7     | 124   | 51      |
| 9    | Palindrome Number                       | Easy   | 8     | 123   | 71      |
| 25   | Reverse Nodes in k-Group                | Hard   | 8     | 122   | 34      |
| 36   | Valid Sudoku                            | Medium | 6     | 121   | 49      |
| 55   | Jump Game                               | Medium | 8     | 121   | 76      |
| 1275 | Find Winner on a Tic Tac Toe Game       | Easy   | 5     | 121   | 50      |
| 43   | Multiply Strings                        | Medium | 6     | 120   | 80      |
| 1004 | Max Consecutive Ones III                | Medium | 7     | 120   | 8       |
| 70   | Climbing Stairs                         | Easy   | 9     | 119   | 59      |
| 46   | Permutations                            | Medium | 9     | 118   | 45      |
| 287  | Find the Duplicate Number               | Medium | 7     | 118   | 6       |
| 101  | Symmetric Tree                          | Easy   | 8     | 117   | 38      |
| 37   | Sudoku Solver                           | Hard   | 11    | 116   | 49      |
| 378  | Kth Smallest Element in a Sorted Matrix | Medium | 8     | 116   | 12      |
| 317  | Shortest Distance from All Buildings †  | Hard   | 4     | 115   | 51      |
| 129  | Sum Root to Leaf Numbers                | Medium | 3     | 114   | 41      |
| 301  | Remove Invalid Parentheses              | Hard   | 2     | 114   | 25      |
| 1146 | Snapshot Array                          | Medium | 2     | 114   | 10      |
| 443  | String Compression                      | Medium | 7     | 111   | 6       |

| #    | PROBLEM                                 | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-----------------------------------------|--------|-------|-------|---------|
| 1539 | Kth Missing Positive Number             | Easy   | 5     | 111   | 10      |
| 658  | Find K Closest Elements                 | Medium | 7     | 110   | 30      |
| 268  | Missing Number                          | Easy   | 8     | 108   | 23      |
| 704  | Binary Search                           | Easy   | 9     | 107   | 10      |
| 136  | Single Number                           | Easy   | 9     | 106   | 23      |
| 588  | Design In-Memory File System †          | Hard   | 6     | 105   | 24      |
| 128  | Longest Consecutive Sequence            | Medium | 9     | 104   | 23      |
| 739  | Daily Temperatures                      | Medium | 5     | 104   | 26      |
| 767  | Reorganize String                       | Medium | 6     | 104   | 21      |
| 35   | Search Insert Position                  | Easy   | 9     | 103   | 10      |
| 605  | Can Place Flowers                       | Easy   | 5     | 103   | 76      |
| 716  | Max Stack †                             | Hard   | 6     | 103   | 24      |
| 122  | Best Time to Buy and Sell Stock II      | Medium | 8     | 102   | 61      |
| 724  | Find Pivot Index                        | Easy   | 9     | 102   | 15      |
| 430  | Flatten a Multilevel Doubly Linked List | Medium | 4     | 100   | 37      |
| 362  | Design Hit Counter †                    | Medium | 7     | 99    | 24      |
| 472  | Concatenated Words                      | Hard   | 2     | 99    | 65      |
| 766  | Toeplitz Matrix                         | Easy   | 2     | 99    | 58      |
| 29   | Divide Two Integers                     | Medium | 7     | 98    | 81      |
| 45   | Jump Game II                            | Medium | 9     | 98    | 76      |
| 84   | Largest Rectangle in Histogram          | Hard   | 8     | 98    | 26      |
| 160  | Intersection of Two Linked Lists        | Easy   | 8     | 97    | 35      |
| 460  | LFU Cache                               | Hard   | 9     | 97    | 24      |
| 364  | Nested List Weight Sum II †             | Medium | 1     | 96    | 51      |
| 539  | Minimum Time Difference                 | Medium | 2     | 96    | 81      |
| 69   | Sqrt(x)                                 | Easy   | 7     | 95    | 10      |
| 169  | Majority Element                        | Easy   | 8     | 95    | 85      |
| 509  | Fibonacci Number                        | Easy   | 9     | 95    | 60      |

| #    | PROBLEM                                             | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-----------------------------------------------------|--------|-------|-------|---------|
| 349  | Intersection of Two Arrays                          | Easy   | 7     | 94    | 5       |
| 706  | Design HashMap                                      | Easy   | 8     | 93    | 24      |
| 1891 | Cutting Ribbons †                                   | Medium | 1     | 93    | 12      |
| 1567 | Maximum Length of Subarray With Positive Product    | Medium | 2     | 91    | 60      |
| 1710 | Maximum Units on a Truck                            | Easy   | 2     | 91    | 75      |
| 2034 | Stock Price Fluctuation                             | Medium | 1     | 90    | 24      |
| 72   | Edit Distance                                       | Medium | 7     | 89    | 63      |
| 24   | Swap Nodes in Pairs                                 | Medium | 6     | 88    | 34      |
| 57   | Insert Interval                                     | Medium | 5     | 87    | 18      |
| 153  | Find Minimum in Rotated Sorted Array                | Medium | 7     | 87    | 10      |
| 410  | Split Array Largest Sum                             | Hard   | 6     | 87    | 13      |
| 1492 | The kth Factor of n                                 | Medium | 2     | 87    | 80      |
| 75   | Sort Colors                                         | Medium | 9     | 86    | 6       |
| 1010 | Pairs of Songs With Total Durations Divisible by 60 | Medium | 7     | 85    | 20      |
| 51   | N-Queens                                            | Hard   | 8     | 84    | 49      |
| 44   | Wildcard Matching                                   | Hard   | 6     | 83    | 63      |
| 981  | Time Based Key-Value Store                          | Medium | 6     | 83    | 24      |
| 99   | Recover Binary Search Tree                          | Medium | 6     | 82    | 39      |
| 219  | Contains Duplicate II                               | Easy   | 7     | 82    | 20      |
| 235  | Lowest Common Ancestor of a Binary Search Tree      | Medium | 6     | 81    | 42      |
| 18   | 4Sum                                                | Medium | 8     | 80    | 5       |
| 647  | Palindromic Substrings                              | Medium | 3     | 80    | 68      |
| 66   | Plus One                                            | Easy   | 6     | 79    | 80      |
| 104  | Maximum Depth of Binary Tree                        | Easy   | 8     | 79    | 38      |
| 953  | Verifying an Alien Dictionary                       | Easy   | 4     | 79    | 54      |
| 73   | Set Matrix Zeroes                                   | Medium | 5     | 78    | 58      |

| #    | PROBLEM                                                                    | LEVEL  | ASKED | SCORE | CHAPTER |
|------|----------------------------------------------------------------------------|--------|-------|-------|---------|
| 398  | Random Pick Index                                                          | Medium | 3     | 78    | 87      |
| 545  | Boundary of Binary Tree †                                                  | Medium | 4     | 78    | 44      |
| 642  | Design Search Autocomplete System †                                        | Hard   | 6     | 78    | 88      |
| 64   | Minimum Path Sum                                                           | Medium | 8     | 77    | 67      |
| 230  | Kth Smallest Element in a BST                                              | Medium | 5     | 77    | 39      |
| 209  | Minimum Size Subarray Sum                                                  | Medium | 7     | 76    | 8       |
| 289  | Game of Life                                                               | Medium | 8     | 76    | 58      |
| 332  | Reconstruct Itinerary                                                      | Hard   | 6     | 76    | 90      |
| 416  | Partition Equal Subset Sum                                                 | Medium | 6     | 76    | 48      |
| 540  | Single Element in a Sorted Array                                           | Medium | 6     | 76    | 10      |
| 1094 | Car Pooling                                                                | Medium | 7     | 76    | 17      |
| 312  | Burst Balloons                                                             | Hard   | 6     | 75    | 68      |
| 167  | Two Sum II - Input Array Is Sorted                                         | Medium | 6     | 74    | 5       |
| 189  | Rotate Array                                                               | Medium | 6     | 74    | 81      |
| 1438 | Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit | Medium | 3     | 73    | 28      |
| 341  | Flatten Nested List Iterator                                               | Medium | 8     | 71    | 88      |
| 32   | Longest Valid Parentheses                                                  | Hard   | 7     | 70    | 25      |
| 63   | Unique Paths II                                                            | Medium | 6     | 70    | 67      |
| 92   | Reverse Linked List II                                                     | Medium | 6     | 70    | 34      |
| 114  | Flatten Binary Tree to Linked List                                         | Medium | 4     | 70    | 44      |
| 240  | Search a 2D Matrix II                                                      | Medium | 6     | 70    | 10      |
| 109  | Convert Sorted List to Binary Search Tree                                  | Medium | 4     | 69    | 42      |
| 221  | Maximal Square                                                             | Medium | 7     | 69    | 27      |
| 282  | Expression Add Operators                                                   | Hard   | 1     | 69    | 45      |
| 2130 | Maximum Twin Sum of a Linked List                                          | Medium | 1     | 69    | 34      |
| 67   | Add Binary                                                                 | Easy   | 6     | 68    | 80      |
| 143  | Reorder List                                                               | Medium | 7     | 68    | 34      |
| 494  | Target Sum                                                                 | Medium | 5     | 67    | 48      |

| #    | PROBLEM                                   | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-------------------------------------------|--------|-------|-------|---------|
| 935  | Knight Dialer                             | Medium | 5     | 67    | 60      |
| 141  | Linked List Cycle                         | Easy   | 7     | 66    | 35      |
| 419  | Battleships in a Board                    | Medium | 5     | 66    | 58      |
| 652  | Find Duplicate Subtrees                   | Medium | 3     | 66    | 44      |
| 1304 | Find N Unique Integers Sum up to Zero     | Easy   | 1     | 66    | 80      |
| 450  | Delete Node in a BST                      | Medium | 5     | 65    | 39      |
| 285  | Inorder Successor in BST †                | Medium | 4     | 64    | 42      |
| 336  | Palindrome Pairs                          | Hard   | 6     | 64    | 89      |
| 370  | Range Addition †                          | Medium | 2     | 64    | 17      |
| 402  | Remove K Digits                           | Medium | 6     | 64    | 26      |
| 1207 | Unique Number of Occurrences              | Easy   | 6     | 64    | 21      |
| 1423 | Maximum Points You Can Obtain from Cards  | Medium | 3     | 64    | 7       |
| 191  | Number of 1 Bits                          | Easy   | 6     | 63    | 78      |
| 270  | Closest Binary Search Tree Value †        | Easy   | 1     | 63    | 42      |
| 1151 | Minimum Swaps to Group All 1's Together † | Medium | 1     | 63    | 7       |
| 344  | Reverse String                            | Easy   | 6     | 62    | 5       |
| 567  | Permutation in String                     | Medium | 7     | 62    | 7       |
| 678  | Valid Parenthesis String                  | Medium | 5     | 62    | 25      |
| 1507 | Reformat Date                             | Easy   | 3     | 62    | 82      |
| 179  | Largest Number                            | Medium | 9     | 61    | 75      |
| 246  | Strobogrammatic Number †                  | Easy   | 2     | 61    | 7       |
| 1344 | Angle Between Hands of a Clock            | Medium | 4     | 61    | 80      |
| 226  | Invert Binary Tree                        | Easy   | 6     | 60    | 38      |
| 277  | Find the Celebrity †                      | Medium | 6     | 60    | 90      |
| 1656 | Design an Ordered Stream                  | Easy   | 1     | 60    | 24      |
| 112  | Path Sum                                  | Easy   | 5     | 59    | 38      |
| 126  | Word Ladder II                            | Hard   | 5     | 59    | 52      |
| 272  | Closest Binary Search Tree Value II †     | Hard   | 2     | 58    | 39      |

| #    | PROBLEM                                                    | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------------|--------|-------|-------|---------|
| 100  | Same Tree                                                  | Easy   | 6     | 57    | 38      |
| 118  | Pascal's Triangle                                          | Easy   | 7     | 57    | 58      |
| 304  | Range Sum Query 2D - Immutable                             | Medium | 5     | 57    | 15      |
| 622  | Design Circular Queue                                      | Medium | 6     | 57    | 37      |
| 876  | Middle of the Linked List                                  | Easy   | 7     | 57    | 35      |
| 1386 | Cinema Seat Allocation                                     | Medium | 2     | 57    | 76      |
| 1428 | Leftmost Column with at Least a One †                      | Medium | 3     | 56    | 10      |
| 1629 | Slowest Key                                                | Easy   | 2     | 55    | 82      |
| 515  | Find Largest Value in Each Tree Row                        | Medium | 1     | 54    | 40      |
| 662  | Maximum Width of Binary Tree                               | Medium | 5     | 54    | 40      |
| 844  | Backspace String Compare                                   | Easy   | 7     | 54    | 5       |
| 852  | Peak Index in a Mountain Array                             | Medium | 4     | 54    | 10      |
| 1046 | Last Stone Weight                                          | Easy   | 4     | 54    | 30      |
| 424  | Longest Repeating Character Replacement                    | Medium | 4     | 53    | 8       |
| 516  | Longest Palindromic Subsequence                            | Medium | 6     | 53    | 68      |
| 148  | Sort List                                                  | Medium | 5     | 51    | 35      |
| 895  | Maximum Frequency Stack                                    | Hard   | 7     | 51    | 32      |
| 1376 | Time Needed to Inform All Employees                        | Medium | 2     | 51    | 90      |
| 311  | Sparse Matrix Multiplication †                             | Medium | 4     | 50    | 58      |
| 529  | Minesweeper                                                | Medium | 5     | 50    | 51      |
| 881  | Boats to Save People                                       | Medium | 7     | 50    | 5       |
| 1366 | Rank Teams by Votes                                        | Medium | 4     | 50    | 82      |
| 1472 | Design Browser History                                     | Medium | 4     | 50    | 24      |
| 93   | Restore IP Addresses                                       | Medium | 6     | 49    | 45      |
| 105  | Construct Binary Tree from Preorder and In-order Traversal | Medium | 6     | 49    | 42      |
| 232  | Implement Queue using Stacks                               | Easy   | 5     | 49    | 29      |
| 323  | Number of Connected Components in an Undirected Graph †    | Medium | 6     | 49    | 51      |

| #    | PROBLEM                                             | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-----------------------------------------------------|--------|-------|-------|---------|
| 490  | The Maze †                                          | Medium | 7     | 49    | 51      |
| 792  | Number of Matching Subsequences                     | Medium | 2     | 49    | 89      |
| 1110 | Delete Nodes And Return Forest                      | Medium | 4     | 49    | 38      |
| 222  | Count Complete Tree Nodes                           | Medium | 3     | 48    | 38      |
| 284  | Peeking Iterator                                    | Medium | 4     | 48    | 88      |
| 328  | Odd Even Linked List                                | Medium | 5     | 48    | 34      |
| 353  | Design Snake Game †                                 | Medium | 6     | 48    | 88      |
| 448  | Find All Numbers Disappeared in an Array            | Easy   | 4     | 48    | 23      |
| 572  | Subtree of Another Tree                             | Easy   | 4     | 48    | 38      |
| 1029 | Two City Scheduling                                 | Medium | 2     | 48    | 75      |
| 1051 | Height Checker                                      | Easy   | 3     | 48    | 33      |
| 1347 | Minimum Number of Steps to Make Two Strings Anagram | Medium | 4     | 48    | 21      |
| 2007 | Find Original Array From Doubled Array              | Medium | 2     | 48    | 75      |
| 243  | Shortest Word Distance †                            | Easy   | 5     | 47    | 82      |
| 432  | All O`one Data Structure                            | Hard   | 3     | 47    | 24      |
| 83   | Remove Duplicates from Sorted List                  | Easy   | 5     | 46    | 34      |
| 113  | Path Sum II                                         | Medium | 5     | 46    | 41      |
| 315  | Count of Smaller Numbers After Self                 | Hard   | 4     | 46    | 85      |
| 417  | Pacific Atlantic Water Flow                         | Medium | 4     | 46    | 49      |
| 542  | 01 Matrix                                           | Medium | 4     | 46    | 52      |
| 694  | Number of Distinct Islands †                        | Medium | 4     | 46    | 51      |
| 274  | H-Index                                             | Medium | 4     | 45    | 33      |
| 403  | Frog Jump                                           | Hard   | 5     | 45    | 56      |
| 463  | Island Perimeter                                    | Easy   | 3     | 45    | 51      |
| 703  | Kth Largest Element in a Stream                     | Easy   | 5     | 45    | 30      |
| 38   | Count and Say                                       | Medium | 6     | 44    | 81      |
| 350  | Intersection of Two Arrays II                       | Easy   | 5     | 44    | 23      |
| 480  | Sliding Window Median                               | Hard   | 3     | 44    | 31      |

| #   | PROBLEM                                                  | LEVEL  | ASKED | SCORE | CHAPTER |
|-----|----------------------------------------------------------|--------|-------|-------|---------|
| 27  | Remove Element                                           | Easy   | 5     | 43    | 6       |
| 97  | Interleaving String                                      | Medium | 5     | 43    | 63      |
| 149 | Max Points on a Line                                     | Hard   | 4     | 43    | 20      |
| 442 | Find All Duplicates in an Array                          | Medium | 5     | 43    | 23      |
| 445 | Add Two Numbers II                                       | Medium | 4     | 43    | 34      |
| 740 | Delete and Earn                                          | Medium | 3     | 43    | 60      |
| 77  | Combinations                                             | Medium | 4     | 42    | 45      |
| 205 | Isomorphic Strings                                       | Easy   | 3     | 42    | 21      |
| 299 | Bulls and Cows                                           | Medium | 2     | 42    | 81      |
| 383 | Ransom Note                                              | Easy   | 4     | 42    | 21      |
| 451 | Sort Characters By Frequency                             | Medium | 5     | 42    | 21      |
| 525 | Contiguous Array                                         | Medium | 4     | 42    | 15      |
| 552 | Student Attendance Record II                             | Hard   | 1     | 42    | 60      |
| 896 | Monotonic Array                                          | Easy   | 2     | 42    | 18      |
| 40  | Combination Sum II                                       | Medium | 5     | 40    | 45      |
| 158 | Read N Characters Given read4 II - Call Multiple Times † | Hard   | 4     | 40    | 92      |
| 161 | One Edit Distance †                                      | Medium | 3     | 40    | 63      |
| 204 | Count Primes                                             | Medium | 5     | 40    | 79      |
| 231 | Power of Two                                             | Easy   | 4     | 40    | 78      |
| 557 | Reverse Words in a String III                            | Easy   | 5     | 40    | 82      |
| 110 | Balanced Binary Tree                                     | Easy   | 5     | 39    | 41      |
| 131 | Palindrome Partitioning                                  | Medium | 5     | 39    | 45      |
| 213 | House Robber II                                          | Medium | 2     | 39    | 60      |
| 325 | Maximum Size Subarray Sum Equals k †                     | Medium | 2     | 39    | 15      |
| 337 | House Robber III                                         | Medium | 3     | 39    | 60      |
| 435 | Non-overlapping Intervals                                | Medium | 3     | 39    | 18      |
| 437 | Path Sum III                                             | Medium | 4     | 39    | 41      |
| 616 | Add Bold Tag in String †                                 | Medium | 1     | 39    | 18      |

| #    | PROBLEM                                   | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-------------------------------------------|--------|-------|-------|---------|
| 856  | Score of Parentheses                      | Medium | 4     | 39    | 25      |
| 1381 | Design a Stack With Increment Operation   | Medium | 1     | 39    | 29      |
| 94   | Binary Tree Inorder Traversal             | Easy   | 4     | 37    | 38      |
| 130  | Surrounded Regions                        | Medium | 4     | 37    | 49      |
| 188  | Best Time to Buy and Sell Stock IV        | Hard   | 5     | 37    | 61      |
| 261  | Graph Valid Tree †                        | Medium | 4     | 37    | 51      |
| 454  | 4Sum II                                   | Medium | 4     | 37    | 20      |
| 617  | Merge Two Binary Trees                    | Easy   | 4     | 37    | 38      |
| 809  | Expressive Words                          | Medium | 3     | 37    | 82      |
| 1143 | Longest Common Subsequence                | Medium | 3     | 37    | 63      |
| 1448 | Count Good Nodes in Binary Tree           | Medium | 2     | 37    | 38      |
| 59   | Spiral Matrix II                          | Medium | 3     | 36    | 58      |
| 190  | Reverse Bits                              | Easy   | 5     | 36    | 78      |
| 257  | Binary Tree Paths                         | Easy   | 5     | 36    | 38      |
| 266  | Palindrome Permutation †                  | Easy   | 1     | 36    | 71      |
| 1352 | Product of the Last K Numbers             | Medium | 3     | 36    | 24      |
| 1603 | Design Parking System                     | Easy   | 1     | 36    | 88      |
| 81   | Search in Rotated Sorted Array II         | Medium | 4     | 35    | 10      |
| 373  | Find K Pairs with Smallest Sums           | Medium | 4     | 35    | 32      |
| 465  | Optimal Account Balancing †               | Hard   | 3     | 34    | 73      |
| 532  | K-diff Pairs in an Array                  | Medium | 7     | 34    | 5       |
| 1487 | Making File Names Unique                  | Medium | 2     | 34    | 82      |
| 30   | Substring with Concatenation of All Words | Hard   | 5     | 33    | 7       |
| 123  | Best Time to Buy and Sell Stock III       | Hard   | 3     | 33    | 61      |
| 165  | Compare Version Numbers                   | Medium | 2     | 33    | 82      |
| 377  | Combination Sum IV                        | Medium | 4     | 33    | 48      |
| 389  | Find the Difference                       | Easy   | 3     | 33    | 78      |
| 535  | Encode and Decode TinyURL                 | Medium | 3     | 33    | 81      |

| #    | PROBLEM                                              | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------|--------|-------|-------|---------|
| 637  | Average of Levels in Binary Tree                     | Easy   | 3     | 33    | 40      |
| 743  | Network Delay Time                                   | Medium | 2     | 33    | 56      |
| 849  | Maximize Distance to Closest Person                  | Medium | 3     | 33    | 18      |
| 958  | Check Completeness of a Binary Tree                  | Medium | 1     | 33    | 40      |
| 1008 | Construct Binary Search Tree from Preorder Traversal | Medium | 4     | 33    | 42      |
| 1305 | All Elements in Two Binary Search Trees              | Medium | 4     | 33    | 38      |
| 1345 | Jump Game IV                                         | Hard   | 3     | 33    | 52      |
| 1382 | Balance a Binary Search Tree                         | Medium | 1     | 33    | 38      |
| 1481 | Least Number of Unique Integers after K Removals     | Medium | 4     | 33    | 21      |
| 698  | Partition to K Equal Sum Subsets                     | Medium | 3     | 32    | 48      |
| 859  | Buddy Strings                                        | Easy   | 2     | 32    | 82      |
| 108  | Convert Sorted Array to Binary Search Tree           | Easy   | 5     | 31    | 42      |
| 157  | Read N Characters Given Read4 †                      | Easy   | 4     | 31    | 92      |
| 203  | Remove Linked List Elements                          | Easy   | 4     | 31    | 37      |
| 723  | Candy Crush †                                        | Medium | 3     | 31    | 58      |
| 1920 | Build Array from Permutation                         | Easy   | 4     | 31    | 58      |
| 82   | Remove Duplicates from Sorted List II                | Medium | 3     | 30    | 34      |
| 171  | Excel Sheet Column Number                            | Easy   | 3     | 30    | 80      |
| 174  | Dungeon Game                                         | Hard   | 2     | 30    | 67      |
| 582  | Kill Process †                                       | Medium | 2     | 30    | 90      |
| 697  | Degree of an Array                                   | Easy   | 5     | 30    | 20      |
| 796  | Rotate String                                        | Easy   | 5     | 30    | 81      |
| 1277 | Count Square Submatrices with All Ones               | Medium | 2     | 30    | 16      |
| 1642 | Furthest Building You Can Reach                      | Medium | 3     | 30    | 75      |
| 1996 | The Number of Weak Characters in the Game            | Medium | 3     | 30    | 26      |
| 729  | My Calendar I                                        | Medium | 3     | 29    | 18      |

| #    | PROBLEM                             | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-------------------------------------|--------|-------|-------|---------|
| 746  | Min Cost Climbing Stairs            | Easy   | 3     | 29    | 60      |
| 1395 | Count Number of Teams               | Medium | 3     | 29    | 81      |
| 220  | Contains Duplicate III              | Hard   | 4     | 28    | 20      |
| 316  | Remove Duplicate Letters            | Medium | 4     | 28    | 26      |
| 468  | Validate IP Address                 | Medium | 3     | 28    | 82      |
| 621  | Task Scheduler                      | Medium | 4     | 28    | 32      |
| 669  | Trim a Binary Search Tree           | Medium | 4     | 28    | 42      |
| 901  | Online Stock Span                   | Medium | 3     | 28    | 26      |
| 979  | Distribute Coins in Binary Tree     | Medium | 4     | 28    | 41      |
| 6    | Zigzag Conversion                   | Medium | 3     | 27    | 81      |
| 85   | Maximal Rectangle                   | Hard   | 3     | 27    | 26      |
| 142  | Linked List Cycle II                | Medium | 2     | 27    | 35      |
| 256  | Paint House †                       | Medium | 3     | 27    | 65      |
| 307  | Range Sum Query - Mutable           | Medium | 2     | 27    | 88      |
| 354  | Russian Doll Envelopes              | Hard   | 2     | 27    | 62      |
| 388  | Longest Absolute File Path          | Medium | 2     | 27    | 29      |
| 420  | Strong Password Checker             | Hard   | 2     | 27    | 82      |
| 488  | Zuma Game                           | Hard   | 3     | 27    | 60      |
| 638  | Shopping Offers                     | Medium | 4     | 27    | 48      |
| 918  | Maximum Sum Circular Subarray       | Medium | 2     | 27    | 61      |
| 1636 | Sort Array by Increasing Frequency  | Easy   | 3     | 27    | 30      |
| 90   | Subsets II                          | Medium | 4     | 25    | 46      |
| 229  | Majority Element II                 | Medium | 3     | 25    | 85      |
| 333  | Largest BST Subtree †               | Medium | 2     | 25    | 41      |
| 358  | Rearrange String k Distance Apart † | Hard   | 4     | 25    | 21      |
| 392  | Is Subsequence                      | Easy   | 3     | 25    | 5       |
| 867  | Transpose Matrix                    | Easy   | 4     | 25    | 58      |
| 135  | Candy                               | Hard   | 1     | 24    | 76      |

| #    | PROBLEM                                            | LEVEL  | ASKED | SCORE | CHAPTER |
|------|----------------------------------------------------|--------|-------|-------|---------|
| 163  | Missing Ranges †                                   | Easy   | 1     | 24    | 18      |
| 166  | Fraction to Recurring Decimal                      | Medium | 3     | 24    | 81      |
| 168  | Excel Sheet Column Title                           | Easy   | 3     | 24    | 80      |
| 214  | Shortest Palindrome                                | Hard   | 3     | 24    | 71      |
| 233  | Number of Digit One                                | Hard   | 2     | 24    | 74      |
| 247  | Strobogrammatic Number II †                        | Medium | 3     | 24    | 25      |
| 279  | Perfect Squares                                    | Medium | 4     | 24    | 60      |
| 390  | Elimination Game                                   | Medium | 3     | 24    | 80      |
| 412  | Fizz Buzz                                          | Easy   | 3     | 24    | 81      |
| 418  | Sentence Screen Fitting †                          | Medium | 1     | 24    | 71      |
| 538  | Convert BST to Greater Tree                        | Medium | 3     | 24    | 39      |
| 581  | Shortest Unsorted Continuous Subarray              | Medium | 2     | 24    | 26      |
| 690  | Employee Importance                                | Medium | 2     | 24    | 52      |
| 946  | Validate Stack Sequences                           | Medium | 3     | 24    | 29      |
| 974  | Subarray Sums Divisible by K                       | Medium | 3     | 24    | 15      |
| 1026 | Maximum Difference Between Node and Ancestor       | Medium | 3     | 24    | 41      |
| 1086 | High Five †                                        | Easy   | 1     | 24    | 32      |
| 1541 | Minimum Insertions to Balance a Parentheses String | Medium | 2     | 24    | 25      |
| 1644 | Lowest Common Ancestor of a Binary Tree II †       | Medium | 1     | 24    | 44      |
| 1721 | Swapping Nodes in a Linked List                    | Medium | 3     | 24    | 34      |
| 1861 | Rotating the Box                                   | Medium | 3     | 24    | 58      |
| 381  | Insert Delete GetRandom O(1) - Duplicates allowed  | Hard   | 2     | 23    | 24      |
| 860  | Lemonade Change                                    | Easy   | 3     | 23    | 76      |
| 218  | The Skyline Problem                                | Hard   | 3     | 22    | 19      |
| 241  | Different Ways to Add Parentheses                  | Medium | 3     | 22    | 85      |
| 671  | Second Minimum Node In a Binary Tree               | Easy   | 1     | 22    | 38      |

| #    | PROBLEM                                            | LEVEL  | ASKED | SCORE | CHAPTER |
|------|----------------------------------------------------|--------|-------|-------|---------|
| 713  | Subarray Product Less Than K                       | Medium | 3     | 22    | 8       |
| 997  | Find the Town Judge                                | Easy   | 4     | 22    | 90      |
| 1512 | Number of Good Pairs                               | Easy   | 3     | 22    | 9       |
| 47   | Permutations II                                    | Medium | 2     | 21    | 47      |
| 61   | Rotate List                                        | Medium | 2     | 21    | 34      |
| 120  | Triangle                                           | Medium | 2     | 21    | 67      |
| 298  | Binary Tree Longest Consecutive Sequence †         | Medium | 3     | 21    | 41      |
| 302  | Smallest Rectangle Enclosing Black Pixels †        | Hard   | 1     | 21    | 51      |
| 338  | Counting Bits                                      | Easy   | 3     | 21    | 78      |
| 536  | Construct Binary Tree from String †                | Medium | 1     | 21    | 44      |
| 640  | Solve the Equation                                 | Medium | 2     | 21    | 81      |
| 643  | Maximum Average Subarray I                         | Easy   | 1     | 21    | 7       |
| 650  | 2 Keys Keyboard                                    | Medium | 2     | 21    | 74      |
| 993  | Cousins in Binary Tree                             | Easy   | 3     | 21    | 38      |
| 1019 | Next Greater Node In Linked List                   | Medium | 2     | 21    | 26      |
| 1287 | Element Appearing More Than 25% In Sorted Array    | Easy   | 2     | 21    | 10      |
| 1614 | Maximum Nesting Depth of the Parentheses           | Easy   | 2     | 21    | 25      |
| 1628 | Design an Expression Tree With Evaluate Function † | Medium | 1     | 21    | 80      |
| 1774 | Closest Dessert Cost                               | Medium | 3     | 21    | 64      |
| 1910 | Remove All Occurrences of a Substring              | Medium | 3     | 21    | 21      |
| 1971 | Find if Path Exists in Graph                       | Easy   | 2     | 21    | 90      |
| 290  | Word Pattern                                       | Easy   | 3     | 20    | 21      |
| 296  | Best Meeting Point †                               | Hard   | 3     | 20    | 91      |
| 360  | Sort Transformed Array †                           | Medium | 1     | 20    | 80      |
| 453  | Minimum Moves to Equal Array Elements              | Medium | 3     | 20    | 80      |
| 744  | Find Smallest Letter Greater Than Target           | Easy   | 3     | 20    | 10      |

| #    | PROBLEM                                                  | LEVEL  | ASKED | SCORE | CHAPTER |
|------|----------------------------------------------------------|--------|-------|-------|---------|
| 1221 | Split a String in Balanced Strings                       | Easy   | 4     | 20    | 82      |
| 1339 | Maximum Product of Splitted Binary Tree                  | Medium | 2     | 20    | 41      |
| 1817 | Finding the Users Active Minutes                         | Medium | 1     | 20    | 20      |
| 355  | Design Twitter                                           | Medium | 3     | 19    | 24      |
| 449  | Serialize and Deserialize BST                            | Medium | 3     | 19    | 42      |
| 771  | Jewels and Stones                                        | Easy   | 3     | 19    | 82      |
| 111  | Minimum Depth of Binary Tree                             | Easy   | 2     | 18    | 38      |
| 115  | Distinct Subsequences                                    | Hard   | 2     | 18    | 63      |
| 156  | Binary Tree Upside Down †                                | Medium | 1     | 18    | 44      |
| 159  | Longest Substring with At Most Two Distinct Characters † | Medium | 2     | 18    | 8       |
| 251  | Flatten 2D Vector †                                      | Medium | 1     | 18    | 88      |
| 252  | Meeting Rooms †                                          | Easy   | 2     | 18    | 18      |
| 313  | Super Ugly Number                                        | Medium | 3     | 18    | 32      |
| 391  | Perfect Rectangle                                        | Hard   | 2     | 18    | 19      |
| 396  | Rotate Function                                          | Medium | 3     | 18    | 81      |
| 414  | Third Maximum Number                                     | Easy   | 3     | 18    | 18      |
| 505  | The Maze II †                                            | Medium | 3     | 18    | 56      |
| 531  | Lonely Pixel I †                                         | Medium | 3     | 18    | 91      |
| 593  | Valid Square                                             | Medium | 2     | 18    | 80      |
| 609  | Find Duplicate File in System                            | Medium | 2     | 18    | 82      |
| 720  | Longest Word in Dictionary                               | Medium | 1     | 18    | 89      |
| 763  | Partition Labels                                         | Medium | 2     | 18    | 75      |
| 788  | Rotated Digits                                           | Medium | 2     | 18    | 74      |
| 814  | Binary Tree Pruning                                      | Medium | 2     | 18    | 41      |
| 841  | Keys and Rooms                                           | Medium | 1     | 18    | 90      |
| 1043 | Partition Array for Maximum Sum                          | Medium | 2     | 18    | 65      |
| 1087 | Brace Expansion †                                        | Medium | 2     | 18    | 82      |
| 1099 | Two Sum Less Than K †                                    | Easy   | 2     | 18    | 5       |

| #    | PROBLEM                                                    | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------------|--------|-------|-------|---------|
| 1267 | Count Servers that Communicate                             | Medium | 1     | 18    | 51      |
| 1372 | Longest ZigZag Path in a Binary Tree                       | Medium | 1     | 18    | 41      |
| 1429 | First Unique Number †                                      | Medium | 2     | 18    | 24      |
| 1588 | Sum of All Odd Length Subarrays                            | Easy   | 3     | 18    | 15      |
| 2016 | Maximum Difference Between Increasing Elements             | Easy   | 2     | 18    | 18      |
| 2038 | Remove Colored Pieces if Both Neighbors are the Same Color | Medium | 1     | 18    | 72      |
| 2055 | Plates Between Candles                                     | Medium | 2     | 18    | 82      |
| 2089 | Find Target Indices After Sorting Array                    | Easy   | 2     | 18    | 10      |
| 2103 | Rings and Rods                                             | Easy   | 1     | 18    | 82      |
| 2125 | Number of Laser Beams in a Bank                            | Medium | 1     | 18    | 58      |
| 319  | Bulb Switcher                                              | Medium | 2     | 17    | 80      |
| 446  | Arithmetic Slices II - Subsequence                         | Hard   | 4     | 17    | 62      |
| 664  | Strange Printer                                            | Hard   | 2     | 17    | 68      |
| 1518 | Water Bottles                                              | Easy   | 3     | 17    | 80      |
| 172  | Factorial Trailing Zeroes                                  | Medium | 2     | 16    | 80      |
| 187  | Repeated DNA Sequences                                     | Medium | 2     | 16    | 7       |
| 305  | Number of Islands II †                                     | Hard   | 2     | 16    | 51      |
| 452  | Minimum Number of Arrows to Burst Balloons                 | Medium | 3     | 16    | 18      |
| 611  | Valid Triangle Number                                      | Medium | 3     | 16    | 5       |
| 701  | Insert into a Binary Search Tree                           | Medium | 3     | 16    | 39      |
| 725  | Split Linked List in Parts                                 | Medium | 3     | 16    | 34      |
| 728  | Self Dividing Numbers                                      | Easy   | 3     | 16    | 80      |
| 1022 | Sum of Root To Leaf Binary Numbers                         | Easy   | 3     | 16    | 38      |
| 1523 | Count Odd Numbers in an Interval Range                     | Easy   | 3     | 16    | 80      |
| 1574 | Shortest Subarray to be Removed to Make Array Sorted       | Medium | 2     | 16    | 26      |

| #    | PROBLEM                                                            | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------------------------|--------|-------|-------|---------|
| 1604 | Alert Using Same Key-Card Three or More Times in a One Hour Period | Medium | 1     | 16    | 82      |
| 1854 | Maximum Population Year                                            | Easy   | 3     | 16    | 15      |
| 1929 | Concatenation of Array                                             | Easy   | 2     | 16    | 58      |
| 58   | Length of Last Word                                                | Easy   | 2     | 15    | 82      |
| 137  | Single Number II                                                   | Medium | 1     | 15    | 23      |
| 237  | Delete Node in a Linked List                                       | Medium | 2     | 15    | 37      |
| 250  | Count Unival Subtrees †                                            | Medium | 2     | 15    | 41      |
| 281  | Zigzag Iterator †                                                  | Medium | 3     | 15    | 88      |
| 334  | Increasing Triplet Subsequence                                     | Medium | 2     | 15    | 62      |
| 335  | Self Crossing                                                      | Hard   | 2     | 15    | 81      |
| 367  | Valid Perfect Square                                               | Easy   | 2     | 15    | 10      |
| 382  | Linked List Random Node                                            | Medium | 3     | 15    | 87      |
| 395  | Longest Substring with At Least K Repeating Characters             | Medium | 2     | 15    | 8       |
| 447  | Number of Boomerangs                                               | Medium | 1     | 15    | 91      |
| 462  | Minimum Moves to Equal Array Elements II                           | Medium | 1     | 15    | 81      |
| 487  | Max Consecutive Ones II †                                          | Medium | 2     | 15    | 8       |
| 551  | Student Attendance Record I                                        | Easy   | 1     | 15    | 82      |
| 562  | Longest Line of Consecutive One in Matrix †                        | Medium | 1     | 15    | 67      |
| 675  | Cut Off Trees for Golf Event                                       | Hard   | 2     | 15    | 52      |
| 676  | Implement Magic Dictionary                                         | Medium | 2     | 15    | 88      |
| 705  | Design HashSet                                                     | Easy   | 2     | 15    | 24      |
| 714  | Best Time to Buy and Sell Stock with Transaction Fee               | Medium | 2     | 15    | 61      |
| 754  | Reach a Number                                                     | Medium | 1     | 15    | 80      |
| 807  | Max Increase to Keep City Skyline                                  | Medium | 2     | 15    | 58      |
| 824  | Goat Latin                                                         | Easy   | 1     | 15    | 82      |
| 884  | Uncommon Words from Two Sentences                                  | Easy   | 2     | 15    | 82      |

| #    | PROBLEM                                       | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-----------------------------------------------|--------|-------|-------|---------|
| 912  | Sort an Array                                 | Medium | 2     | 15    | 33      |
| 980  | Unique Paths III                              | Hard   | 2     | 15    | 49      |
| 1291 | Sequential Digits                             | Medium | 2     | 15    | 81      |
| 1313 | Decompress Run-Length Encoded List            | Easy   | 2     | 15    | 18      |
| 1331 | Rank Transform of an Array                    | Easy   | 2     | 15    | 20      |
| 1360 | Number of Days Between Two Dates              | Easy   | 1     | 15    | 81      |
| 1408 | String Matching in an Array                   | Easy   | 2     | 15    | 82      |
| 1417 | Reformat The String                           | Easy   | 2     | 15    | 82      |
| 1446 | Consecutive Characters                        | Easy   | 2     | 15    | 82      |
| 1470 | Shuffle the Array                             | Easy   | 2     | 15    | 18      |
| 1514 | Path with Maximum Probability                 | Medium | 1     | 15    | 56      |
| 1658 | Minimum Operations to Reduce X to Zero        | Medium | 2     | 15    | 8       |
| 2076 | Process Restricted Friend Requests            | Hard   | 1     | 15    | 57      |
| 374  | Guess Number Higher or Lower                  | Easy   | 2     | 14    | 10      |
| 628  | Maximum Product of Three Numbers              | Easy   | 1     | 14    | 81      |
| 756  | Pyramid Transition Matrix                     | Medium | 3     | 14    | 45      |
| 757  | Set Intersection Size At Least Two            | Hard   | 1     | 14    | 75      |
| 781  | Rabbits in Forest                             | Medium | 3     | 14    | 76      |
| 799  | Champagne Tower                               | Medium | 3     | 14    | 81      |
| 1009 | Complement of Base 10 Integer                 | Easy   | 2     | 14    | 78      |
| 1257 | Smallest Common Region †                      | Medium | 1     | 14    | 44      |
| 1389 | Create Target Array in the Given Order        | Easy   | 3     | 14    | 58      |
| 1400 | Construct K Palindrome Strings                | Medium | 1     | 14    | 71      |
| 1700 | Number of Students Unable to Eat Lunch        | Easy   | 1     | 14    | 29      |
| 1823 | Find the Winner of the Circular Game          | Medium | 2     | 14    | 24      |
| 265  | Paint House II †                              | Hard   | 3     | 13    | 65      |
| 271  | Encode and Decode Strings †                   | Medium | 2     | 13    | 82      |
| 1353 | Maximum Number of Events That Can Be Attended | Medium | 2     | 13    | 75      |

| #    | PROBLEM                                                | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------------|--------|-------|-------|---------|
| 1641 | Count Sorted Vowel Strings                             | Medium | 2     | 13    | 80      |
| 95   | Unique Binary Search Trees II                          | Medium | 1     | 12    | 42      |
| 96   | Unique Binary Search Trees                             | Medium | 1     | 12    | 42      |
| 164  | Maximum Gap                                            | Medium | 2     | 12    | 33      |
| 186  | Reverse Words in a String II †                         | Medium | 1     | 12    | 82      |
| 248  | Strobogrammatic Number III †                           | Hard   | 1     | 12    | 25      |
| 255  | Verify Preorder Sequence in Binary Search Tree †       | Medium | 2     | 12    | 44      |
| 288  | Unique Word Abbreviation †                             | Medium | 2     | 12    | 88      |
| 292  | Nim Game                                               | Easy   | 2     | 12    | 72      |
| 340  | Longest Substring with At Most K Distinct Characters † | Medium | 2     | 12    | 8       |
| 351  | Android Unlock Patterns †                              | Medium | 2     | 12    | 73      |
| 405  | Convert a Number to Hexadecimal                        | Easy   | 2     | 12    | 78      |
| 407  | Trapping Rain Water II                                 | Hard   | 1     | 12    | 32      |
| 413  | Arithmetic Slices                                      | Medium | 2     | 12    | 7       |
| 456  | 132 Pattern                                            | Medium | 1     | 12    | 26      |
| 459  | Repeated Substring Pattern                             | Easy   | 2     | 12    | 82      |
| 475  | Heaters                                                | Medium | 2     | 12    | 81      |
| 476  | Number Complement                                      | Easy   | 2     | 12    | 78      |
| 485  | Max Consecutive Ones                                   | Easy   | 2     | 12    | 18      |
| 520  | Detect Capital                                         | Easy   | 2     | 12    | 82      |
| 524  | Longest Word in Dictionary through Deleting            | Medium | 1     | 12    | 82      |
| 549  | Binary Tree Longest Consecutive Sequence II †          | Medium | 1     | 12    | 41      |
| 566  | Reshape the Matrix                                     | Easy   | 2     | 12    | 58      |
| 576  | Out of Boundary Paths                                  | Medium | 2     | 12    | 67      |
| 633  | Sum of Square Numbers                                  | Medium | 2     | 12    | 80      |
| 655  | Print Binary Tree                                      | Medium | 2     | 12    | 38      |

| #    | PROBLEM                                                | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------------|--------|-------|-------|---------|
| 659  | Split Array into Consecutive Subsequences              | Medium | 2     | 12    | 81      |
| 673  | Number of Longest Increasing Sub-sequence              | Medium | 2     | 12    | 62      |
| 674  | Longest Continuous Increasing Sub-sequence             | Easy   | 1     | 12    | 18      |
| 684  | Redundant Connection                                   | Medium | 2     | 12    | 55      |
| 687  | Longest Univalve Path                                  | Medium | 2     | 12    | 41      |
| 755  | Pour Water †                                           | Medium | 1     | 12    | 58      |
| 783  | Minimum Distance Between BST Nodes                     | Easy   | 2     | 12    | 39      |
| 821  | Shortest Distance to a Character                       | Easy   | 2     | 12    | 82      |
| 904  | Fruit Into Baskets                                     | Medium | 1     | 12    | 8       |
| 929  | Unique Email Addresses                                 | Easy   | 1     | 12    | 82      |
| 1002 | Find Common Characters                                 | Easy   | 1     | 12    | 21      |
| 1024 | Video Stitching                                        | Medium | 2     | 12    | 76      |
| 1025 | Divisor Game                                           | Easy   | 2     | 12    | 60      |
| 1038 | Binary Search Tree to Greater Sum Tree                 | Medium | 2     | 12    | 39      |
| 1066 | Campus Bikes II †                                      | Medium | 2     | 12    | 73      |
| 1079 | Letter Tile Possibilities                              | Medium | 2     | 12    | 45      |
| 1100 | Find K-Length Substrings With No Repeated Characters † | Medium | 1     | 12    | 7       |
| 1108 | Defanging an IP Address                                | Easy   | 2     | 12    | 82      |
| 1160 | Find Words That Can Be Formed by Characters            | Easy   | 2     | 12    | 21      |
| 1200 | Minimum Absolute Difference                            | Easy   | 2     | 12    | 18      |
| 1282 | Group the People Given the Group Size They Belong To   | Medium | 2     | 12    | 76      |
| 1296 | Divide Array in Sets of K Consecutive Numbers          | Medium | 1     | 12    | 62      |
| 1332 | Remove Palindromic Subsequences                        | Easy   | 1     | 12    | 71      |
| 1342 | Number of Steps to Reduce a Number to Zero             | Easy   | 2     | 12    | 78      |

| #    | PROBLEM                                                          | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------------------|--------|-------|-------|---------|
| 1348 | Tweet Counts Per Frequency                                       | Medium | 1     | 12    | 32      |
| 1358 | Number of Substrings Containing All Three Characters             | Medium | 3     | 12    | 9       |
| 1367 | Linked List in Binary Tree                                       | Medium | 2     | 12    | 41      |
| 1377 | Frog Position After T Seconds                                    | Hard   | 2     | 12    | 81      |
| 1451 | Rearrange Words in a Sentence                                    | Medium | 3     | 12    | 82      |
| 1464 | Maximum Product of Two Elements in an Array                      | Easy   | 3     | 12    | 32      |
| 1524 | Number of Sub-arrays With Odd Sum                                | Medium | 2     | 12    | 80      |
| 1526 | Minimum Number of Increments on Subarrays to Form a Target Array | Hard   | 1     | 12    | 17      |
| 1557 | Minimum Number of Vertices to Reach All Nodes                    | Medium | 2     | 12    | 54      |
| 1560 | Most Visited Sector in a Circular Track                          | Easy   | 1     | 12    | 58      |
| 1572 | Matrix Diagonal Sum                                              | Easy   | 2     | 12    | 58      |
| 1592 | Rearrange Spaces Between Words                                   | Easy   | 1     | 12    | 82      |
| 1711 | Count Good Meals                                                 | Medium | 3     | 12    | 20      |
| 1763 | Longest Nice Substring                                           | Easy   | 1     | 12    | 85      |
| 1765 | Map of Highest Peak                                              | Medium | 2     | 12    | 51      |
| 1773 | Count Items Matching a Rule                                      | Easy   | 2     | 12    | 82      |
| 1832 | Check if the Sentence Is Pangram                                 | Easy   | 2     | 12    | 82      |
| 1985 | Find the Kth Largest Integer in the Array                        | Medium | 1     | 12    | 30      |
| 2018 | Check if Word Can Be Placed In Crossword                         | Medium | 1     | 12    | 81      |
| 2110 | Number of Smooth Descent Periods of a Stock                      | Medium | 1     | 12    | 60      |
| 527  | Word Abbreviation †                                              | Hard   | 2     | 11    | 89      |
| 1289 | Minimum Falling Path Sum II                                      | Hard   | 2     | 11    | 67      |
| 60   | Permutation Sequence                                             | Hard   | 2     | 10    | 47      |
| 119  | Pascal's Triangle II                                             | Easy   | 2     | 10    | 58      |
| 144  | Binary Tree Preorder Traversal                                   | Easy   | 2     | 10    | 38      |

| #    | PROBLEM                                              | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------|--------|-------|-------|---------|
| 245  | Shortest Word Distance III †                         | Medium | 2     | 10    | 82      |
| 254  | Factor Combinations †                                | Medium | 1     | 10    | 32      |
| 291  | Word Pattern II †                                    | Medium | 2     | 10    | 45      |
| 310  | Minimum Height Trees                                 | Medium | 2     | 10    | 52      |
| 379  | Design Phone Directory †                             | Medium | 2     | 10    | 24      |
| 393  | UTF-8 Validation                                     | Medium | 2     | 10    | 78      |
| 421  | Maximum XOR of Two Numbers in an Array               | Medium | 2     | 10    | 78      |
| 477  | Total Hamming Distance                               | Medium | 2     | 10    | 78      |
| 486  | Predict the Winner                                   | Medium | 2     | 10    | 72      |
| 502  | IPO                                                  | Hard   | 1     | 10    | 31      |
| 591  | Tag Validator                                        | Hard   | 2     | 10    | 25      |
| 592  | Fraction Addition and Subtraction                    | Medium | 1     | 10    | 81      |
| 599  | Minimum Index Sum of Two Lists                       | Easy   | 2     | 10    | 82      |
| 635  | Design Log Storage System †                          | Medium | 1     | 10    | 24      |
| 646  | Maximum Length of Pair Chain                         | Medium | 2     | 10    | 62      |
| 653  | Two Sum IV - Input is a BST                          | Easy   | 2     | 10    | 20      |
| 836  | Rectangle Overlap                                    | Easy   | 2     | 10    | 19      |
| 888  | Fair Candy Swap                                      | Easy   | 2     | 10    | 10      |
| 923  | 3Sum With Multiplicity                               | Medium | 2     | 10    | 5       |
| 1071 | Greatest Common Divisor of Strings                   | Easy   | 2     | 10    | 80      |
| 1190 | Reverse Substrings Between Each Pair of Parentheses  | Medium | 2     | 10    | 25      |
| 1281 | Subtract the Product and Sum of Digits of an Integer | Easy   | 1     | 10    | 80      |
| 1283 | Find the Smallest Divisor Given a Threshold          | Medium | 2     | 10    | 12      |
| 1290 | Convert Binary Number in a Linked List to Integer    | Easy   | 2     | 10    | 34      |
| 1365 | How Many Numbers Are Smaller Than the Current Number | Easy   | 2     | 10    | 33      |
| 1749 | Maximum Absolute Sum of Any Subarray                 | Medium | 2     | 10    | 61      |

| #    | PROBLEM                                          | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------|--------|-------|-------|---------|
| 1836 | Remove Duplicates From an Unsorted Linked List † | Medium | 2     | 10    | 81      |
| 1859 | Sorting the Sentence                             | Easy   | 2     | 10    | 82      |
| 216  | Combination Sum III                              | Medium | 1     | 9     | 45      |
| 225  | Implement Stack using Queues                     | Easy   | 1     | 9     | 29      |
| 263  | Ugly Number                                      | Easy   | 1     | 9     | 80      |
| 267  | Palindrome Permutation II †                      | Medium | 1     | 9     | 45      |
| 308  | Range Sum Query 2D - Mutable †                   | Medium | 1     | 9     | 16      |
| 321  | Create Maximum Number                            | Hard   | 2     | 9     | 26      |
| 343  | Integer Break                                    | Medium | 1     | 9     | 60      |
| 400  | Nth Digit                                        | Medium | 1     | 9     | 80      |
| 406  | Queue Reconstruction by Height                   | Medium | 1     | 9     | 81      |
| 422  | Valid Word Square †                              | Easy   | 2     | 9     | 58      |
| 425  | Word Squares †                                   | Hard   | 1     | 9     | 89      |
| 429  | N-ary Tree Level Order Traversal                 | Medium | 1     | 9     | 38      |
| 466  | Count The Repetitions                            | Hard   | 1     | 9     | 82      |
| 493  | Reverse Pairs                                    | Hard   | 1     | 9     | 85      |
| 504  | Base 7                                           | Easy   | 1     | 9     | 81      |
| 514  | Freedom Trail                                    | Hard   | 2     | 9     | 82      |
| 530  | Minimum Absolute Difference in BST               | Easy   | 1     | 9     | 39      |
| 548  | Split Array with Equal Sum †                     | Hard   | 1     | 9     | 15      |
| 565  | Array Nesting                                    | Medium | 2     | 9     | 51      |
| 589  | N-ary Tree Preorder Traversal                    | Easy   | 1     | 9     | 38      |
| 631  | Design Excel Sum Formula †                       | Hard   | 1     | 9     | 24      |
| 665  | Non-decreasing Array                             | Medium | 1     | 9     | 18      |
| 666  | Path Sum IV †                                    | Medium | 2     | 9     | 43      |
| 688  | Knight Probability in Chessboard                 | Medium | 1     | 9     | 67      |
| 689  | Maximum Sum of 3 Non-Overlapping Subarrays       | Hard   | 1     | 9     | 81      |

| #    | PROBLEM                                                | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------------|--------|-------|-------|---------|
| 691  | Stickers to Spell Word                                 | Hard   | 1     | 9     | 73      |
| 719  | Find K-th Smallest Pair Distance                       | Hard   | 1     | 9     | 14      |
| 734  | Sentence Similarity †                                  | Easy   | 2     | 9     | 82      |
| 738  | Monotone Increasing Digits                             | Medium | 2     | 9     | 76      |
| 742  | Closest Leaf in a Binary Tree †                        | Medium | 1     | 9     | 38      |
| 776  | Split BST †                                            | Medium | 1     | 9     | 42      |
| 830  | Positions of Large Groups                              | Easy   | 1     | 9     | 82      |
| 832  | Flipping an Image                                      | Easy   | 1     | 9     | 58      |
| 840  | Magic Squares In Grid                                  | Medium | 1     | 9     | 58      |
| 900  | RLE Iterator                                           | Medium | 1     | 9     | 88      |
| 954  | Array of Doubled Pairs                                 | Medium | 1     | 9     | 75      |
| 965  | Univalued Binary Tree                                  | Easy   | 1     | 9     | 38      |
| 1119 | Remove Vowels from a String †                          | Easy   | 1     | 9     | 82      |
| 1137 | N-th Tribonacci Number                                 | Easy   | 1     | 9     | 60      |
| 1161 | Maximum Level Sum of a Binary Tree                     | Medium | 1     | 9     | 38      |
| 1213 | Intersection of Three Sorted Arrays †                  | Easy   | 1     | 9     | 20      |
| 1214 | Two Sum BSTs †                                         | Medium | 1     | 9     | 42      |
| 1258 | Synonymous Sentences †                                 | Medium | 1     | 9     | 45      |
| 1266 | Minimum Time Visiting All Points                       | Easy   | 1     | 9     | 91      |
| 1373 | Maximum Sum BST in Binary Tree                         | Hard   | 1     | 9     | 42      |
| 1431 | Kids With the Greatest Number of Candies               | Easy   | 1     | 9     | 18      |
| 1466 | Reorder Routes to Make All Paths Lead to the City Zero | Medium | 1     | 9     | 52      |
| 1583 | Count Unhappy Friends                                  | Medium | 1     | 9     | 58      |
| 1662 | Check If Two String Arrays are Equivalent              | Easy   | 2     | 9     | 82      |
| 1670 | Design Front Middle Back Queue                         | Medium | 1     | 9     | 24      |
| 1680 | Concatenation of Consecutive Binary Numbers            | Medium | 1     | 9     | 78      |

| #    | PROBLEM                                                    | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------------|--------|-------|-------|---------|
| 1718 | Construct the Lexicographically Largest Valid Sequence     | Medium | 2     | 9     | 45      |
| 1732 | Find the Highest Altitude                                  | Easy   | 1     | 9     | 15      |
| 1877 | Minimize Maximum Pair Sum in Array                         | Medium | 1     | 9     | 81      |
| 1974 | Minimum Time to Type Word Using Special Typewriter         | Easy   | 1     | 9     | 75      |
| 1991 | Find the Middle Index in Array                             | Easy   | 1     | 9     | 15      |
| 2033 | Minimum Operations to Make a Uni-Value Grid                | Medium | 1     | 9     | 80      |
| 2090 | K Radius Subarray Averages                                 | Medium | 1     | 9     | 7       |
| 361  | Bomb Enemy †                                               | Medium | 1     | 8     | 67      |
| 365  | Water and Jug Problem                                      | Medium | 1     | 8     | 80      |
| 436  | Find Right Interval                                        | Medium | 2     | 8     | 18      |
| 811  | Subdomain Visit Count                                      | Medium | 2     | 8     | 21      |
| 1150 | Check If a Number Is Majority Element in a Sorted Array †  | Easy   | 2     | 8     | 12      |
| 1232 | Check If It Is a Straight Line                             | Easy   | 2     | 8     | 91      |
| 1705 | Maximum Number of Eaten Apples                             | Medium | 2     | 8     | 32      |
| 2050 | Parallel Courses III                                       | Hard   | 1     | 8     | 54      |
| 484  | Find Permutation †                                         | Medium | 2     | 7     | 47      |
| 683  | K Empty Slots †                                            | Hard   | 2     | 7     | 24      |
| 1128 | Number of Equivalent Domino Pairs                          | Easy   | 2     | 7     | 20      |
| 1544 | Make The String Great                                      | Easy   | 2     | 7     | 29      |
| 1601 | Maximum Number of Achievable Transfer Requests             | Hard   | 2     | 7     | 50      |
| 1844 | Replace All Digits with Characters                         | Easy   | 2     | 7     | 82      |
| 52   | N-Queens II                                                | Hard   | 1     | 6     | 49      |
| 80   | Remove Duplicates from Sorted Array II                     | Medium | 1     | 6     | 6       |
| 89   | Gray Code                                                  | Medium | 1     | 6     | 45      |
| 106  | Construct Binary Tree from Inorder and Postorder Traversal | Medium | 1     | 6     | 42      |

| #   | PROBLEM                                 | LEVEL  | ASKED | SCORE | CHAPTER |
|-----|-----------------------------------------|--------|-------|-------|---------|
| 107 | Binary Tree Level Order Traversal II    | Medium | 1     | 6     | 40      |
| 132 | Palindrome Partitioning II              | Hard   | 1     | 6     | 71      |
| 145 | Binary Tree Postorder Traversal         | Easy   | 1     | 6     | 38      |
| 147 | Insertion Sort List                     | Medium | 1     | 6     | 36      |
| 154 | Find Minimum in Rotated Sorted Array II | Hard   | 1     | 6     | 10      |
| 223 | Rectangle Area                          | Medium | 1     | 6     | 19      |
| 258 | Add Digits                              | Easy   | 1     | 6     | 80      |
| 260 | Single Number III                       | Medium | 1     | 6     | 23      |
| 264 | Ugly Number II                          | Medium | 1     | 6     | 32      |
| 276 | Paint Fence †                           | Medium | 1     | 6     | 81      |
| 280 | Wiggle Sort †                           | Medium | 1     | 6     | 75      |
| 303 | Range Sum Query - Immutable             | Easy   | 1     | 6     | 15      |
| 306 | Additive Number                         | Medium | 1     | 6     | 45      |
| 318 | Maximum Product of Word Lengths         | Medium | 1     | 6     | 78      |
| 326 | Power of Three                          | Easy   | 1     | 6     | 80      |
| 330 | Patching Array                          | Hard   | 1     | 6     | 76      |
| 345 | Reverse Vowels of a String              | Easy   | 1     | 6     | 5       |
| 352 | Data Stream as Disjoint Intervals       | Hard   | 1     | 6     | 18      |
| 357 | Count Numbers with Unique Digits        | Medium | 1     | 6     | 45      |
| 368 | Largest Divisible Subset                | Medium | 1     | 6     | 62      |
| 371 | Sum of Two Integers                     | Medium | 1     | 6     | 78      |
| 375 | Guess Number Higher or Lower II         | Medium | 1     | 6     | 68      |
| 376 | Wiggle Subsequence                      | Medium | 1     | 6     | 76      |
| 397 | Integer Replacement                     | Medium | 1     | 6     | 60      |
| 409 | Longest Palindrome                      | Easy   | 1     | 6     | 71      |
| 434 | Number of Segments in a String          | Easy   | 1     | 6     | 82      |
| 440 | K-th Smallest in Lexicographical Order  | Hard   | 1     | 6     | 89      |
| 444 | Sequence Reconstruction †               | Medium | 1     | 6     | 54      |

| #   | PROBLEM                                | LEVEL  | ASKED | SCORE | CHAPTER |
|-----|----------------------------------------|--------|-------|-------|---------|
| 455 | Assign Cookies                         | Easy   | 1     | 6     | 75      |
| 461 | Hamming Distance                       | Easy   | 1     | 6     | 78      |
| 464 | Can I Win                              | Medium | 1     | 6     | 48      |
| 467 | Unique Substrings in Wraparound String | Medium | 1     | 6     | 71      |
| 471 | Encode String with Shortest Length †   | Hard   | 1     | 6     | 71      |
| 473 | Matchsticks to Square                  | Medium | 1     | 6     | 48      |
| 474 | Ones and Zeroes                        | Medium | 1     | 6     | 64      |
| 483 | Smallest Good Base                     | Hard   | 1     | 6     | 80      |
| 492 | Construct the Rectangle                | Easy   | 1     | 6     | 80      |
| 499 | The Maze III †                         | Hard   | 1     | 6     | 56      |
| 508 | Most Frequent Subtree Sum              | Medium | 1     | 6     | 38      |
| 517 | Super Washing Machines                 | Hard   | 1     | 6     | 76      |
| 522 | Longest Uncommon Subsequence II        | Medium | 1     | 6     | 82      |
| 526 | Beautiful Arrangement                  | Medium | 1     | 6     | 47      |
| 541 | Reverse String II                      | Easy   | 1     | 6     | 82      |
| 554 | Brick Wall                             | Medium | 1     | 6     | 20      |
| 559 | Maximum Depth of N-ary Tree            | Easy   | 1     | 6     | 38      |
| 568 | Maximum Vacation Days †                | Hard   | 1     | 6     | 67      |
| 583 | Delete Operation for Two Strings       | Medium | 1     | 6     | 63      |
| 590 | N-ary Tree Postorder Traversal         | Easy   | 1     | 6     | 38      |
| 598 | Range Addition II                      | Easy   | 1     | 6     | 17      |
| 604 | Design Compressed String Iterator †    | Easy   | 1     | 6     | 24      |
| 606 | Construct String from Binary Tree      | Medium | 1     | 6     | 38      |
| 629 | K Inverse Pairs Array                  | Hard   | 1     | 6     | 81      |
| 630 | Course Schedule III                    | Hard   | 1     | 6     | 76      |
| 634 | Find the Derangement of An Array †     | Medium | 1     | 6     | 80      |
| 639 | Decode Ways II                         | Hard   | 1     | 6     | 60      |
| 645 | Set Mismatch                           | Easy   | 1     | 6     | 23      |

| #   | PROBLEM                        | LEVEL  | ASKED | SCORE | CHAPTER |
|-----|--------------------------------|--------|-------|-------|---------|
| 648 | Replace Words                  | Medium | 1     | 6     | 89      |
| 651 | 4 Keys Keyboard †              | Medium | 1     | 6     | 74      |
| 654 | Maximum Binary Tree            | Medium | 1     | 6     | 26      |
| 660 | Remove 9 †                     | Hard   | 1     | 6     | 81      |
| 661 | Image Smoother                 | Easy   | 1     | 6     | 16      |
| 663 | Equal Tree Partition †         | Medium | 1     | 6     | 41      |
| 677 | Map Sum Pairs                  | Medium | 1     | 6     | 89      |
| 679 | 24 Game                        | Hard   | 1     | 6     | 48      |
| 685 | Redundant Connection II        | Hard   | 1     | 6     | 55      |
| 686 | Repeated String Match          | Medium | 1     | 6     | 82      |
| 700 | Search in a Binary Search Tree | Easy   | 1     | 6     | 38      |
| 709 | To Lower Case                  | Easy   | 1     | 6     | 82      |
| 717 | 1-bit and 2-bit Characters     | Easy   | 1     | 6     | 18      |
| 727 | Minimum Window Subsequence †   | Hard   | 1     | 6     | 8       |
| 737 | Sentence Similarity II †       | Medium | 1     | 6     | 55      |
| 748 | Shortest Completing Word       | Easy   | 1     | 6     | 82      |
| 758 | Bold Words in String †         | Medium | 1     | 6     | 18      |
| 764 | Largest Plus Sign              | Medium | 1     | 6     | 65      |
| 765 | Couples Holding Hands          | Hard   | 1     | 6     | 55      |
| 769 | Max Chunks To Make Sorted      | Medium | 1     | 6     | 26      |
| 775 | Global and Local Inversions    | Medium | 1     | 6     | 80      |
| 784 | Letter Case Permutation        | Medium | 1     | 6     | 45      |
| 819 | Most Common Word               | Easy   | 1     | 6     | 82      |
| 820 | Short Encoding of Words        | Medium | 1     | 6     | 89      |
| 823 | Binary Trees With Factors      | Medium | 1     | 6     | 81      |
| 838 | Push Dominoes                  | Medium | 1     | 6     | 82      |
| 861 | Score After Flipping Matrix    | Medium | 1     | 6     | 58      |
| 870 | Advantage Shuffle              | Medium | 1     | 6     | 81      |

| #    | PROBLEM                                                | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------------|--------|-------|-------|---------|
| 872  | Leaf-Similar Trees                                     | Easy   | 1     | 6     | 38      |
| 877  | Stone Game                                             | Medium | 1     | 6     | 72      |
| 880  | Decoded String at Index                                | Medium | 1     | 6     | 29      |
| 890  | Find and Replace Pattern                               | Medium | 1     | 6     | 22      |
| 892  | Surface Area of 3D Shapes                              | Easy   | 1     | 6     | 91      |
| 897  | Increasing Order Search Tree                           | Easy   | 1     | 6     | 38      |
| 914  | X of a Kind in a Deck of Cards                         | Easy   | 1     | 6     | 80      |
| 917  | Reverse Only Letters                                   | Easy   | 1     | 6     | 82      |
| 922  | Sort Array By Parity II                                | Easy   | 1     | 6     | 6       |
| 925  | Long Pressed Name                                      | Easy   | 1     | 6     | 82      |
| 931  | Minimum Falling Path Sum                               | Medium | 1     | 6     | 67      |
| 941  | Valid Mountain Array                                   | Easy   | 1     | 6     | 18      |
| 942  | DI String Match                                        | Easy   | 1     | 6     | 82      |
| 951  | Flip Equivalent Binary Trees                           | Medium | 1     | 6     | 44      |
| 970  | Powerful Integers                                      | Medium | 1     | 6     | 80      |
| 976  | Largest Perimeter Triangle                             | Easy   | 1     | 6     | 80      |
| 988  | Smallest String Starting From Leaf                     | Medium | 1     | 6     | 41      |
| 989  | Add to Array-Form of Integer                           | Easy   | 1     | 6     | 80      |
| 1005 | Maximize Sum Of Array After K Negations                | Easy   | 1     | 6     | 75      |
| 1014 | Best Sightseeing Pair                                  | Medium | 1     | 6     | 65      |
| 1021 | Remove Outermost Parentheses                           | Easy   | 1     | 6     | 25      |
| 1056 | Confusing Number †                                     | Easy   | 1     | 6     | 80      |
| 1057 | Campus Bikes †                                         | Medium | 1     | 6     | 33      |
| 1114 | Print in Order                                         | Easy   | 1     | 6     | 92      |
| 1122 | Relative Sort Array                                    | Easy   | 1     | 6     | 33      |
| 1136 | Parallel Courses †                                     | Medium | 1     | 6     | 54      |
| 1138 | Alphabet Board Path                                    | Medium | 1     | 6     | 82      |
| 1170 | Compare Strings by Frequency of the Smallest Character | Medium | 1     | 6     | 82      |

| #    | PROBLEM                                                | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------------|--------|-------|-------|---------|
| 1171 | Remove Zero Sum Consecutive Nodes from Linked List     | Medium | 1     | 6     | 15      |
| 1175 | Prime Arrangements                                     | Easy   | 1     | 6     | 80      |
| 1185 | Day of the Week                                        | Easy   | 1     | 6     | 80      |
| 1219 | Path with Maximum Gold                                 | Medium | 1     | 6     | 49      |
| 1237 | Find Positive Integer Solution for a Given Equation    | Medium | 1     | 6     | 10      |
| 1252 | Cells with Odd Values in a Matrix                      | Easy   | 1     | 6     | 80      |
| 1260 | Shift 2D Grid                                          | Easy   | 1     | 6     | 58      |
| 1295 | Find Numbers with Even Number of Digits                | Easy   | 1     | 6     | 80      |
| 1297 | Maximum Number of Occurrences of a Substring           | Medium | 1     | 6     | 21      |
| 1299 | Replace Elements with Greatest Element on Right Side   | Easy   | 1     | 6     | 18      |
| 1300 | Sum of Mutated Array Closest to Target                 | Medium | 1     | 6     | 12      |
| 1302 | Deepest Leaves Sum                                     | Medium | 1     | 6     | 38      |
| 1314 | Matrix Block Sum                                       | Medium | 1     | 6     | 15      |
| 1315 | Sum of Nodes with Even-Valued Grandparent              | Medium | 1     | 6     | 38      |
| 1325 | Delete Leaves With a Given Value                       | Medium | 1     | 6     | 41      |
| 1329 | Sort the Matrix Diagonally                             | Medium | 1     | 6     | 58      |
| 1338 | Reduce Array Size to The Half                          | Medium | 1     | 6     | 33      |
| 1346 | Check If N and Its Double Exist                        | Easy   | 1     | 6     | 10      |
| 1351 | Count Negative Numbers in a Sorted Matrix              | Easy   | 1     | 6     | 10      |
| 1354 | Construct Target Array With Multiple Sums              | Hard   | 1     | 6     | 32      |
| 1361 | Validate Binary Tree Nodes                             | Medium | 1     | 6     | 55      |
| 1370 | Increasing Decreasing String                           | Easy   | 1     | 6     | 21      |
| 1374 | Generate a String With Characters That Have Odd Counts | Easy   | 1     | 6     | 82      |

| #    | PROBLEM                                                            | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------------------------|--------|-------|-------|---------|
| 1379 | Find a Corresponding Node of a Binary Tree in a Clone of That Tree | Easy   | 1     | 6     | 38      |
| 1380 | Lucky Numbers in a Matrix                                          | Easy   | 1     | 6     | 58      |
| 1387 | Sort Integers by The Power Value                                   | Medium | 1     | 6     | 60      |
| 1388 | Pizza With 3n Slices                                               | Hard   | 1     | 6     | 81      |
| 1392 | Longest Happy Prefix                                               | Hard   | 1     | 6     | 83      |
| 1403 | Minimum Subsequence in Non-Increasing Order                        | Easy   | 1     | 6     | 75      |
| 1413 | Minimum Value to Get Positive Step by Step Sum                     | Easy   | 1     | 6     | 15      |
| 1436 | Destination City                                                   | Easy   | 1     | 6     | 82      |
| 1439 | Find the Kth Smallest Sum of a Matrix With Sorted Rows             | Hard   | 1     | 6     | 14      |
| 1441 | Build an Array With Stack Operations                               | Medium | 1     | 6     | 29      |
| 1447 | Simplified Fractions                                               | Medium | 1     | 6     | 80      |
| 1455 | Check If a Word Occurs As a Prefix of Any Word in a Sentence       | Easy   | 1     | 6     | 15      |
| 1457 | Pseudo-Palindromic Paths in a Binary Tree                          | Medium | 1     | 6     | 41      |
| 1475 | Final Prices With a Special Discount in a Shop                     | Easy   | 1     | 6     | 26      |
| 1480 | Running Sum of 1d Array                                            | Easy   | 1     | 6     | 15      |
| 1482 | Minimum Number of Days to Make m Bouquets                          | Medium | 1     | 6     | 13      |
| 1485 | Clone Binary Tree With Random Pointer †                            | Medium | 1     | 6     | 38      |
| 1486 | XOR Operation in an Array                                          | Easy   | 1     | 6     | 78      |
| 1490 | Clone N-ary Tree †                                                 | Medium | 1     | 6     | 38      |
| 1491 | Average Salary Excluding the Minimum and Maximum Salary            | Easy   | 1     | 6     | 18      |
| 1496 | Path Crossing                                                      | Easy   | 1     | 6     | 82      |
| 1525 | Number of Good Ways to Split a String                              | Medium | 1     | 6     | 82      |
| 1528 | Shuffle String                                                     | Easy   | 1     | 6     | 82      |

| #    | PROBLEM                                                    | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------------|--------|-------|-------|---------|
| 1534 | Count Good Triplets                                        | Easy   | 1     | 6     | 81      |
| 1551 | Minimum Operations to Make Array Equal                     | Medium | 1     | 6     | 80      |
| 1561 | Maximum Number of Coins You Can Get                        | Medium | 1     | 6     | 75      |
| 1576 | Replace All ?'s to Avoid Consecutive Repeating Characters  | Easy   | 1     | 6     | 82      |
| 1624 | Largest Substring Between Two Equal Characters             | Easy   | 1     | 6     | 82      |
| 1626 | Best Team With No Conflicts                                | Medium | 2     | 6     | 62      |
| 1630 | Arithmetic Subarrays                                       | Medium | 1     | 6     | 81      |
| 1640 | Check Array Formation Through Concatenation                | Easy   | 1     | 6     | 20      |
| 1664 | Ways to Make a Fair Array                                  | Medium | 1     | 6     | 15      |
| 1673 | Find the Most Competitive Subsequence                      | Medium | 1     | 6     | 26      |
| 1675 | Minimize Deviation in Array                                | Hard   | 1     | 6     | 32      |
| 1678 | Goal Parser Interpretation                                 | Easy   | 1     | 6     | 82      |
| 1679 | Max Number of K-Sum Pairs                                  | Medium | 1     | 6     | 20      |
| 1689 | Partitioning Into Minimum Number Of Deci-Binary Numbers    | Medium | 1     | 6     | 76      |
| 1742 | Maximum Number of Balls in a Box                           | Easy   | 1     | 6     | 80      |
| 1748 | Sum of Unique Elements                                     | Easy   | 1     | 6     | 81      |
| 1752 | Check if Array Is Sorted and Rotated                       | Easy   | 1     | 6     | 18      |
| 1753 | Maximum Score From Removing Stones                         | Medium | 1     | 6     | 80      |
| 1768 | Merge Strings Alternately                                  | Easy   | 1     | 6     | 6       |
| 1769 | Minimum Number of Operations to Move All Balls to Each Box | Medium | 1     | 6     | 15      |
| 1772 | Sort Features by Popularity †                              | Medium | 1     | 6     | 82      |
| 1775 | Equal Sum Arrays With Minimum Number of Operations         | Medium | 1     | 6     | 76      |
| 1779 | Find Nearest Point That Has the Same X or Y Coordinate     | Easy   | 1     | 6     | 18      |

| #    | PROBLEM                                                       | LEVEL  | ASKED | SCORE | CHAPTER |
|------|---------------------------------------------------------------|--------|-------|-------|---------|
| 1780 | Check if Number is a Sum of Powers of Three                   | Medium | 1     | 6     | 80      |
| 1790 | Check if One String Swap Can Make Strings Equal               | Easy   | 1     | 6     | 23      |
| 1792 | Maximum Average Pass Ratio                                    | Medium | 1     | 6     | 32      |
| 1804 | Implement Trie II (Prefix Tree) †                             | Medium | 1     | 6     | 24      |
| 1807 | Evaluate the Bracket Pairs of a String                        | Medium | 1     | 6     | 25      |
| 1814 | Count Nice Pairs in an Array                                  | Medium | 1     | 6     | 80      |
| 1827 | Minimum Operations to Make the Array Increasing               | Easy   | 1     | 6     | 76      |
| 1869 | Longer Contiguous Segments of Ones than Zeros                 | Easy   | 1     | 6     | 82      |
| 1909 | Remove One Element to Make the Array Strictly Increasing      | Easy   | 1     | 6     | 18      |
| 1925 | Count Square Sum Triples                                      | Easy   | 1     | 6     | 63      |
| 1926 | Nearest Exit from Entrance in Maze                            | Medium | 1     | 6     | 52      |
| 1968 | Array With Elements Not Equal to Average of Neighbors         | Medium | 1     | 6     | 75      |
| 1980 | Find Unique Binary String                                     | Medium | 1     | 6     | 45      |
| 2023 | Number of Pairs of Strings With Concatenation Equal to Target | Medium | 1     | 6     | 21      |
| 2024 | Maximize the Confusion of an Exam                             | Medium | 1     | 6     | 8       |
| 2042 | Check if Numbers Are Ascending in a Sentence                  | Easy   | 1     | 6     | 82      |
| 2047 | Number of Valid Words in a Sentence                           | Easy   | 1     | 6     | 82      |
| 2078 | Two Furthest Houses With Different Colors                     | Easy   | 1     | 6     | 76      |
| 2116 | Check if a Parentheses String Can Be Valid                    | Medium | 1     | 6     | 25      |
| 2134 | Minimum Swaps to Group All 1's Together II                    | Medium | 1     | 6     | 7       |
| 2166 | Design Bitset                                                 | Medium | 1     | 6     | 88      |
| 2259 | Remove Digit From Number to Maximize Result                   | Easy   | 1     | 6     | 76      |

| #    | PROBLEM                                    | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------|--------|-------|-------|---------|
| 86   | Partition List                             | Medium | 1     | 4     | 34      |
| 87   | Scramble String                            | Hard   | 1     | 4     | 68      |
| 170  | Two Sum III - Data structure design †      | Easy   | 1     | 4     | 20      |
| 201  | Bitwise AND of Numbers Range               | Medium | 1     | 4     | 78      |
| 275  | H-Index II                                 | Medium | 1     | 4     | 10      |
| 324  | Wiggle Sort II                             | Medium | 1     | 4     | 86      |
| 372  | Super Pow                                  | Medium | 1     | 4     | 80      |
| 404  | Sum of Left Leaves                         | Easy   | 1     | 4     | 38      |
| 457  | Circular Array Loop                        | Medium | 1     | 4     | 6       |
| 495  | Teemo Attacking                            | Easy   | 1     | 4     | 18      |
| 507  | Perfect Number                             | Easy   | 1     | 4     | 80      |
| 513  | Find Bottom Left Tree Value                | Medium | 1     | 4     | 40      |
| 546  | Remove Boxes                               | Hard   | 1     | 4     | 68      |
| 682  | Baseball Game                              | Easy   | 1     | 4     | 29      |
| 779  | K-th Symbol in Grammar                     | Medium | 1     | 4     | 80      |
| 804  | Unique Morse Code Words                    | Easy   | 1     | 4     | 82      |
| 848  | Shifting Letters                           | Medium | 1     | 4     | 15      |
| 885  | Spiral Matrix III                          | Medium | 1     | 4     | 58      |
| 950  | Reveal Cards In Increasing Order           | Medium | 1     | 4     | 24      |
| 999  | Available Captures for Rook                | Easy   | 1     | 4     | 58      |
| 1062 | Longest Repeating Substring †              | Medium | 1     | 4     | 84      |
| 1103 | Distribute Candies to People               | Easy   | 1     | 4     | 80      |
| 1182 | Shortest Distance to Target Color †        | Medium | 1     | 4     | 12      |
| 1189 | Maximum Number of Balloons                 | Easy   | 1     | 4     | 21      |
| 1198 | Find Smallest Common Element in All Rows † | Medium | 1     | 4     | 81      |
| 1228 | Missing Number In Arithmetic Progression † | Easy   | 1     | 4     | 80      |
| 1286 | Iterator for Combination                   | Medium | 1     | 4     | 88      |

| #    | PROBLEM                                              | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------|--------|-------|-------|---------|
| 1385 | Find the Distance Value Between Two Arrays           | Easy   | 1     | 4     | 6       |
| 1401 | Circle and Rectangle Overlapping                     | Medium | 1     | 4     | 91      |
| 1556 | Thousand Separator                                   | Easy   | 1     | 4     | 82      |
| 1582 | Special Positions in a Binary Matrix                 | Easy   | 1     | 4     | 58      |
| 1663 | Smallest String With A Given Numeric Value           | Medium | 1     | 4     | 76      |
| 1672 | Richest Customer Wealth                              | Easy   | 1     | 4     | 58      |
| 1688 | Count of Matches in Tournament                       | Easy   | 1     | 4     | 80      |
| 1733 | Minimum Number of People to Teach                    | Medium | 1     | 4     | 75      |
| 1758 | Minimum Changes To Make Alternating Binary String    | Easy   | 1     | 4     | 82      |
| 1805 | Number of Different Integers in a String             | Easy   | 1     | 4     | 82      |
| 1863 | Sum of All Subset XOR Totals                         | Easy   | 1     | 4     | 45      |
| 1886 | Determine Whether Matrix Can Be Obtained By Rotation | Easy   | 1     | 4     | 58      |
| 2073 | Time Needed to Buy Tickets                           | Easy   | 1     | 4     | 24      |
| 293  | Flip Game †                                          | Easy   | 1     | 3     | 82      |
| 294  | Flip Game II †                                       | Medium | 1     | 3     | 72      |
| 320  | Generalized Abbreviation †                           | Medium | 1     | 3     | 45      |
| 331  | Verify Preorder Serialization of a Binary Tree       | Medium | 1     | 3     | 44      |
| 369  | Plus One Linked List †                               | Medium | 1     | 3     | 34      |
| 401  | Binary Watch                                         | Easy   | 1     | 3     | 45      |
| 411  | Minimum Unique Word Abbreviation †                   | Hard   | 1     | 3     | 82      |
| 469  | Convex Polygon †                                     | Medium | 1     | 3     | 91      |
| 481  | Magical String                                       | Medium | 1     | 3     | 82      |
| 482  | License Key Formatting                               | Easy   | 1     | 3     | 82      |
| 506  | Relative Ranks                                       | Easy   | 1     | 3     | 32      |
| 521  | Longest Uncommon Subsequence I                       | Easy   | 1     | 3     | 82      |
| 533  | Lonely Pixel II †                                    | Medium | 1     | 3     | 91      |

| #    | PROBLEM                                           | LEVEL  | ASKED | SCORE | CHAPTER |
|------|---------------------------------------------------|--------|-------|-------|---------|
| 537  | Complex Number Multiplication                     | Medium | 1     | 3     | 80      |
| 553  | Optimal Division                                  | Medium | 1     | 3     | 74      |
| 555  | Split Concatenated Strings †                      | Medium | 1     | 3     | 76      |
| 587  | Erect the Fence                                   | Hard   | 1     | 3     | 91      |
| 625  | Minimum Factorization †                           | Medium | 1     | 3     | 76      |
| 644  | Maximum Average Subarray II †                     | Hard   | 1     | 3     | 13      |
| 656  | Coin Path †                                       | Hard   | 1     | 3     | 65      |
| 667  | Beautiful Arrangement II                          | Medium | 1     | 3     | 80      |
| 672  | Bulb Switcher II                                  | Medium | 1     | 3     | 78      |
| 750  | Number Of Corner Rectangles †                     | Medium | 1     | 3     | 91      |
| 760  | Find Anagram Mappings †                           | Easy   | 1     | 3     | 21      |
| 762  | Prime Number of Set Bits in Binary Representation | Easy   | 1     | 3     | 78      |
| 789  | Escape The Ghosts                                 | Medium | 1     | 3     | 80      |
| 800  | Similar RGB Color †                               | Easy   | 1     | 3     | 80      |
| 812  | Largest Triangle Area                             | Easy   | 1     | 3     | 81      |
| 816  | Ambiguous Coordinates                             | Medium | 1     | 3     | 82      |
| 893  | Groups of Special-Equivalent Strings              | Medium | 1     | 3     | 82      |
| 933  | Number of Recent Calls                            | Easy   | 1     | 3     | 24      |
| 944  | Delete Columns to Make Sorted                     | Easy   | 1     | 3     | 62      |
| 1020 | Number of Enclaves                                | Medium | 1     | 3     | 49      |
| 1033 | Moving Stones Until Consecutive                   | Medium | 1     | 3     | 80      |
| 1037 | Valid Boomerang                                   | Easy   | 1     | 3     | 91      |
| 1065 | Index Pairs of a String †                         | Easy   | 1     | 3     | 89      |
| 1078 | Occurrences After Bigram                          | Easy   | 1     | 3     | 21      |
| 1085 | Sum of Digits in the Minimum Number †             | Easy   | 1     | 3     | 80      |
| 1090 | Largest Values From Labels                        | Medium | 1     | 3     | 75      |
| 1118 | Number of Days in a Month †                       | Easy   | 1     | 3     | 80      |
| 1133 | Largest Unique Number †                           | Easy   | 1     | 3     | 81      |

| #    | PROBLEM                                                                 | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-------------------------------------------------------------------------|--------|-------|-------|---------|
| 1134 | Armstrong Number †                                                      | Easy   | 1     | 3     | 80      |
| 1165 | Single-Row Keyboard †                                                   | Easy   | 1     | 3     | 82      |
| 1176 | Diet Plan Performance †                                                 | Easy   | 1     | 3     | 7       |
| 1184 | Distance Between Bus Stops                                              | Easy   | 1     | 3     | 18      |
| 1261 | Find Elements in a Contaminated Binary Tree                             | Medium | 1     | 3     | 38      |
| 1273 | Delete Tree Nodes †                                                     | Medium | 1     | 3     | 41      |
| 1324 | Print Words Vertically                                                  | Medium | 1     | 3     | 82      |
| 1337 | The K Weakest Rows in a Matrix                                          | Easy   | 1     | 3     | 10      |
| 1357 | Apply Discount Every n Orders                                           | Medium | 1     | 3     | 88      |
| 1362 | Closest Divisors                                                        | Medium | 1     | 3     | 80      |
| 1371 | Find the Longest Substring Containing Vowels in Even Counts             | Medium | 1     | 3     | 15      |
| 1394 | Find Lucky Integer in an Array                                          | Easy   | 1     | 3     | 81      |
| 1409 | Queries on a Permutation With Key                                       | Medium | 1     | 3     | 81      |
| 1415 | The k-th Lexicographical String of All Happy Strings of Length n        | Medium | 1     | 3     | 45      |
| 1422 | Maximum Score After Splitting a String                                  | Easy   | 1     | 3     | 15      |
| 1452 | People Whose List of Favorite Companies Is Not a Subset of Another List | Medium | 1     | 3     | 82      |
| 1456 | Maximum Number of Vowels in a Substring of Given Length                 | Medium | 1     | 3     | 7       |
| 1461 | Check If a String Contains All Binary Codes of Size K                   | Medium | 1     | 3     | 84      |
| 1469 | Find All The Lonely Nodes †                                             | Easy   | 1     | 3     | 38      |
| 1471 | The k Strongest Values in an Array                                      | Medium | 1     | 3     | 5       |
| 1474 | Delete N Nodes After M Nodes of a Linked List †                         | Easy   | 1     | 3     | 37      |
| 1502 | Can Make Arithmetic Progression From Sequence                           | Easy   | 1     | 3     | 18      |
| 1508 | Range Sum of Sorted Subarray Sums                                       | Medium | 1     | 3     | 14      |

| #    | PROBLEM                                                      | LEVEL  | ASKED | SCORE | CHAPTER |
|------|--------------------------------------------------------------|--------|-------|-------|---------|
| 1558 | Minimum Numbers of Function Calls to Make Target Array       | Medium | 1     | 3     | 76      |
| 1608 | Special Array With X Elements Greater Than or Equal X        | Easy   | 1     | 3     | 10      |
| 1609 | Even Odd Tree                                                | Medium | 1     | 3     | 40      |
| 1619 | Mean of Array After Removing Some Elements                   | Easy   | 1     | 3     | 18      |
| 1637 | Widest Vertical Area Between Two Points Containing No Points | Easy   | 1     | 3     | 75      |
| 1708 | Largest Subarray Length K †                                  | Easy   | 1     | 3     | 76      |
| 1717 | Maximum Score From Removing Substrings                       | Medium | 1     | 3     | 26      |
| 1726 | Tuple with Same Product                                      | Medium | 1     | 3     | 81      |
| 1736 | Latest Time by Replacing Hidden Digits                       | Easy   | 1     | 3     | 76      |
| 1756 | Design Most Recently Used Queue †                            | Medium | 1     | 3     | 24      |
| 1764 | Form Array by Concatenating Subarrays of Another Array       | Medium | 1     | 3     | 82      |
| 1781 | Sum of Beauty of All Substrings                              | Medium | 1     | 3     | 21      |
| 1791 | Find Center of Star Graph                                    | Easy   | 1     | 3     | 54      |
| 1806 | Minimum Number of Operations to Reinitialize a Permutation   | Medium | 1     | 3     | 58      |
| 1813 | Sentence Similarity III                                      | Medium | 1     | 3     | 82      |
| 1816 | Truncate Sentence                                            | Easy   | 1     | 3     | 82      |
| 1826 | Faulty Sensor †                                              | Easy   | 1     | 3     | 6       |
| 1828 | Queries on Number of Points Inside a Circle                  | Medium | 1     | 3     | 91      |
| 1860 | Incremental Memory Leak                                      | Medium | 1     | 3     | 80      |
| 1874 | Minimize Product Sum of Two Arrays †                         | Medium | 1     | 3     | 76      |
| 1893 | Check if All the Integers in a Range Are Covered             | Easy   | 1     | 3     | 17      |
| 1945 | Sum of Digits of String After Convert                        | Easy   | 1     | 3     | 82      |
| 1952 | Three Divisors                                               | Easy   | 1     | 3     | 80      |

| #    | PROBLEM                                                    | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------------|--------|-------|-------|---------|
| 1966 | Binary Searchable Numbers in an Unsorted Array †           | Medium | 1     | 3     | 26      |
| 1981 | Minimize the Difference Between Target and Chosen Elements | Medium | 1     | 3     | 64      |
| 2006 | Count Number of Pairs With Absolute Difference K           | Easy   | 1     | 3     | 81      |
| 2012 | Sum of Beauty in the Array                                 | Medium | 1     | 3     | 18      |
| 2022 | Convert 1D Array Into 2D Array                             | Easy   | 1     | 3     | 58      |
| 2053 | Kth Distinct String in an Array                            | Easy   | 1     | 3     | 21      |
| 2057 | Smallest Index With Equal Value                            | Easy   | 1     | 3     | 18      |
| 2075 | Decode the Slanted Ciphertext                              | Medium | 1     | 3     | 82      |
| 2079 | Watering Plants                                            | Medium | 1     | 3     | 58      |
| 2109 | Adding Spaces to a String                                  | Medium | 1     | 3     | 82      |
| 2114 | Maximum Number of Words Found in Sentences                 | Easy   | 1     | 3     | 82      |
| 2124 | Check if All A's Appears Before All B's                    | Easy   | 1     | 3     | 82      |
| 2126 | Destroying Asteroids                                       | Medium | 1     | 3     | 75      |
| 2149 | Rearrange Array Elements by Sign                           | Medium | 1     | 3     | 5       |
| 2150 | Find All Lonely Numbers in the Array                       | Medium | 1     | 3     | 81      |
| 2155 | All Divisions With the Highest Score of a Binary Array     | Medium | 1     | 3     | 18      |
| 2160 | Minimum Sum of Four Digit Number After Splitting Digits    | Easy   | 1     | 3     | 75      |
| 2161 | Partition Array According to Given Pivot                   | Medium | 1     | 3     | 81      |
| 2181 | Merge Nodes in Between Zeros                               | Medium | 1     | 3     | 34      |
| 2185 | Counting Words With a Given Prefix                         | Easy   | 1     | 3     | 83      |
| 2190 | Most Frequent Number Following Key In an Array             | Easy   | 1     | 3     | 81      |
| 2200 | Find All K-Distant Indices in an Array                     | Easy   | 1     | 3     | 6       |
| 2206 | Divide Array Into Equal Pairs                              | Easy   | 1     | 3     | 81      |
| 2208 | Minimum Operations to Halve Array Sum                      | Medium | 1     | 3     | 30      |

| #    | PROBLEM                                                                       | LEVEL  | ASKED | SCORE | CHAPTER |
|------|-------------------------------------------------------------------------------|--------|-------|-------|---------|
| 2210 | Count Hills and Valleys in an Array                                           | Easy   | 1     | 3     | 18      |
| 2215 | Find the Difference of Two Arrays                                             | Easy   | 1     | 3     | 23      |
| 2244 | Minimum Rounds to Complete All Tasks                                          | Medium | 1     | 3     | 81      |
| 2255 | Count Prefixes of a Given String                                              | Easy   | 1     | 3     | 82      |
| 2256 | Minimum Average Difference                                                    | Medium | 1     | 3     | 15      |
| 2260 | Minimum Consecutive Cards to Pick Up                                          | Medium | 1     | 3     | 23      |
| 385  | Mini Parser                                                                   | Medium | 1     | 2     | 25      |
| 439  | Ternary Expression Parser †                                                   | Medium | 1     | 2     | 25      |
| 479  | Largest Palindrome Product                                                    | Hard   | 1     | 2     | 81      |
| 573  | Squirrel Simulation †                                                         | Medium | 1     | 2     | 81      |
| 624  | Maximum Distance in Arrays                                                    | Medium | 1     | 2     | 76      |
| 693  | Binary Number with Alternating Bits                                           | Easy   | 1     | 2     | 78      |
| 908  | Smallest Range I                                                              | Easy   | 1     | 2     | 80      |
| 1030 | Matrix Cells in Distance Order                                                | Easy   | 1     | 2     | 91      |
| 1343 | Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold | Medium | 1     | 2     | 7       |
| 1410 | HTML Entity Parser                                                            | Medium | 1     | 2     | 82      |
| 1427 | Perform String Shifts †                                                       | Easy   | 1     | 2     | 81      |
| 1754 | Largest Merge Of Two Strings                                                  | Medium | 1     | 2     | 82      |
| 1784 | Check if Binary String Has at Most One Segment of Ones                        | Easy   | 1     | 2     | 82      |
| 1785 | Minimum Elements to Add to Form a Given Sum                                   | Medium | 1     | 2     | 76      |
| 1797 | Design Authentication Manager                                                 | Medium | 1     | 2     | 24      |
| 1829 | Maximum XOR for Each Query                                                    | Medium | 1     | 2     | 15      |
| 1845 | Seat Reservation Manager                                                      | Medium | 1     | 2     | 88      |
| 1876 | Substrings of Size Three with Distinct Characters                             | Easy   | 1     | 2     | 7       |
| 1935 | Maximum Number of Words You Can Type                                          | Easy   | 1     | 2     | 82      |
| 1961 | Check If String Is a Prefix of Array                                          | Easy   | 1     | 2     | 15      |

| #    | PROBLEM                                              | LEVEL  | ASKED | SCORE | CHAPTER |
|------|------------------------------------------------------|--------|-------|-------|---------|
| 1967 | Number of Strings That Appear as Sub-strings in Word | Easy   | 1     | 2     | 82      |
| 2032 | Two Out of Three                                     | Easy   | 1     | 2     | 23      |
| 2043 | Simple Bank System                                   | Medium | 1     | 2     | 88      |
| 2068 | Check Whether Two Strings are Almost Equivalent      | Easy   | 1     | 2     | 21      |
| 2080 | Range Frequency Queries                              | Medium | 1     | 2     | 10      |
| 2085 | Count Common Words With One Occurrence               | Easy   | 1     | 2     | 21      |
| 2154 | Keep Multiplying Found Values by Two                 | Easy   | 1     | 2     | 81      |
| 2243 | Calculate Digit Sum of a String                      | Easy   | 1     | 2     | 82      |
| 2248 | Intersection of Multiple Arrays                      | Easy   | 1     | 2     | 81      |